IN THE UNITED STATES COURT OF INTERNATIONAL TRADE
BEFORE THE HONORABLE JUDGES THEREOF

COMPLAINT AND PETITION

for Declaratory, Mandamus, and Injunctive Relief

WITH ATTACHED

Patent Specification, PCT Filing Materials,
Post-Quantum Cryptographic Integration Blueprints,
Complete Diagram Sets (FIGS 1–32 + CIT FIGS 1–4),
and Manufacturing Designs

CAUSTIN LEE MCLAUGHLIN,
Plaintiff, Pro Se

v.

UNITED STATES OF AMERICA;
UNITED STATES PATENT AND TRADEMARK OFFICE;
DEPARTMENT OF COMMERCE;
DEPARTMENT OF HOMELAND SECURITY;
and JOHN AND JANE DOE FEDERAL ACTORS 1–5,
Defendants.

Civil Action No.: CIT 26-00_____
DEMAND FOR JURY TRIAL

Document ID	CIT-2026-DAG-PQC-001
Revision	1.0 (Unified)
Architecture	DAG-only; no Merkle constructions
ISA Coverage	RISC-V (RV32I / RV64GC / RV128I); x86 (32/64); ARMv7 (A32/T32); ARMv8 (AArch64/AArch32); L
Figures	32 HSPA + 4 CIT + 4 Open Diamond
Manufacturing	Diamond probe stack, ZU9EG 10-layer PCB, GDSII mask layers, BOM
License	CC-BY-SA-4.0 (Doc), Apache-2.0 / GPL-3.0 (Code)
Date	October 29, 2026

This document compiles with a standard TeX Live distribution. All figures are ASCII renderings inside verbatim environments. Cryptographic integrity is realized exclusively through content-addressed DAG structure. No Merkle tree, Merkle root, or Merkle proof appears anywhere in this filing.

Contents

I Complaint and Petition (CIT Pleading) 7

1 Nature of the Action 8

2 Jurisdiction and Venue 9

3 Parties 10

4 Statement of Facts 11

5 Prayer for Relief 12

6 Verification and Certificate of Service 13

II Open Diamond Electrophoretic Neuroprosthetic Platform 14

7 Executive Summary 15
7.1 Design Principles . . . 15

8 Physical Layer Stack and Cross-Section 16

9 Non-Electrical Superposition Physics 17
9.1 Superposition Region . . . 17
9.2 Activation Equation . . . 17

10 Lidar Subsystem 18

11 CSF-Distributed Nanotechnology 19

12 Pure-R Driver Ecosystem (Reference Implementation) 20

III Q128 Unified Processor: ISA and Multi-ISA Backwards Compatibility 21

13 Q128 Base ISA 22

14 Backwards-Compatible Execution Modes 23

15 Full Multi-ISA Specification (Reference) 24

IV HSPA v3.0 Semantic Processing (Figures 1–22) 25

16 Master Figure-to-Claim Matrix 26

17 FIG 1: HSPA Core System Architecture 27

18 FIG 2: 128-Bit SEG Node Bit Structure **28**

19 FIG 3: PMCU Bayesian Inference and Dynamic Scheduler **30**

20 FIG 4: Unified Memory Lattice **31**

21 FIG 5: SOML Safe-Commit Protocol **32**

22 FIG 6: Asymmetric N-Refinement Dual-Clock HTC **33**

23 FIG 7: LoongDiamond-2.5 Quantum Substrate **35**

24 FIG 8: PQC Multi-Tier Trust Chain (DAG-Only) **36**

25 FIG 9: ZU9EG Carrier Board PCB Stackup (Manufacturing Design) **37**

26 FIG 10: HARMONI-X Accessibility Pipeline **38**

27 FIGS 11–22: Extended Subsystems **39**

V Full Multi-ISA Backwards Compatibility (Figures 23–32) 40

28 Multi-ISA Overview **41**

29 FIG 23: Multi-ISA Decode Front-End **42**

30 FIG 24: Cross-ISA Register File Aliasing **44**

31 FIG 25: Memory Consistency Translation **45**

32 FIG 26: Syscall Dispatcher and ABI Translation **47**

33 FIG 27: Dynamic Binary Translation Cache **49**

34 FIG 28: Self-Modifying Code Detection **50**

35 FIG 29: LoongArch Decode and Execution Path **51**

36 FIG 30: LoongArch Register Mapping **53**

37 FIG 31: LoongArch Memory Model and DBAR Translation **55**

38 FIG 32: LoongArch LSX / LASX Vector Translation **56**

VI DAG-Only HAL, Driver, and IDE Architecture (CIT Figures 1–4) 57

39 Field and Background **58**
39.1 Field of the Invention 58
39.2 Background 58
39.3 Summary 58

40 DAG Node Taxonomy and Formal Mathematics **59**
40.1 Algebraic Transformation Rules 59

41 CIT FIG 1: Pure DAG Node Taxonomy and Synchronization Topology **60**

42 CIT FIG 2: Graph Transformation Engine **61**

43 CIT FIG 3: HAL DAG-Aware DMA Descriptor Chain (Manufacturing) 62

44 CIT FIG 4: IDE Build Pipeline and Driver Fault Recovery 63

45 WIPO ST.26 Compliant XML Schema 64

46 Source Code: HAL MMIO Driver 65

47 Source Code: IDE DAG Scheduler (Python) 66

VII Post-Quantum Cryptographic Integration 68

48 FIPS 203/204/205 Primitives and QRDE-AEAD 69
48.1 Overview . . . 69
48.2 Mathematical Foundations . . . 69
48.2.1 Module Learning With Errors (M-LWE) . . . 69
48.2.2 Module Short Integer Solution (M-SIS) . . . 69
48.3 QRDE-AEAD Dual-Envelope Protocol . . . 69
48.4 Post-Quantum Cryptographic Integration Code (C) . . . 69

VIII Manufacturing Designs 71

49 Diamond CVD Fabrication Sequence 72

50 GDSII Mask Layer Outline 73

51 Bill of Materials (Device) 74

52 Regulatory and Test Flow 75

IX USPTO Claims (1–28) 76

53 Independent Claims 77

54 Open Diamond and Q128 Claims 78

55 HSPA Processor Claims (9–18) 79

56 Extended Claims (19–28) 80

X Patent Eligibility and Formal Verification 81

57 Patent Eligibility (35 U.S.C. § 101 / MPEP 2106) 82
57.1 Step 1 — Statutory Category . . . 82
57.2 Step 2A, Prong 1 — Abstract Concept . . . 82
57.3 Step 2A, Prong 2 — Practical Application . . . 82
57.4 Step 2B — Inventive Concept . . . 82

58 Formal Verification Programme 83

A Glossary 84

B Complete Figure Index 86

List of Figures

8.1 Probe layer cross-section (manufacturing view). 16

10.1 Lidar range–Doppler processing DAG. 18

11.1 Nanoparticle administration and tracking DAG. 19

17.1 FIG. 1 — HSPA Core System Architecture. 27

18.1 FIG. 2 — SEG node dataflow and DAG dependency queue. 28

19.1 FIG. 3 — PMCU Bayesian inference and dynamic scheduler. 30

20.1 FIG. 4 — UML hybrid address decoding. 31

21.1 FIG. 5 — SOML safe-commit protocol. 32

22.1 FIG. 6 — Dual-clock HTC and convergence trigger. 34

23.1 FIG. 7 — Density-matrix quantum substrate. 35

24.1 FIG. 8 — PQC multi-tier trust chain with DAG-only anchoring. 36

26.1 FIG. 10 — HARMONI-X accessibility pipeline. 38

29.1 FIG. 23 — Multi-ISA decode front-end. 43

30.1 FIG. 24 — Cross-ISA register file aliasing. 44

31.1 FIG. 25 — Memory consistency translation rules. 46

32.1 FIG. 26 — Syscall dispatcher and ABI translation. 48

33.1 FIG. 27 — Dynamic binary translation cache. 49

34.1 FIG. 28 — Self-modifying code detection. 50

35.1 FIG. 29 — LoongArch decode and execution path. 52

36.1 FIG. 30 — LoongArch register mapping to Q128 physical registers. 54

37.1 FIG. 31 — LoongArch memory model and DBAR translation. 55

38.1 FIG. 32 — LoongArch LSX/LASX vector translation. 56

41.1 CIT FIG. 1 — DAG node taxonomy and topology. 60

42.1 CIT FIG. 2 — DAG dependency transformer. 61

43.1 CIT FIG. 3 — HAL DAG-aware DMA descriptor chain. 62

44.1 CIT FIG. 4 — IDE/DWARF integration and fault recovery. 63

List of Tables

8.1 Open Diamond probe layer stack (manufacturing design). 16

13.1 Q128 instruction categories. 22

14.1 Supported ISA execution modes. 23

16.1 Master figure-to-claim matrix (FIGS 1–22). 26

18.1 128-bit SEG node bit field allocation. 29

20.1 UML 128-bit hybrid address structure. 31

25.1 10-layer PCB stackup (revision 1.0). 37
25.2 Power rail management (sequencing design). 37

28.1 Dimensions of full ISA backwards compatibility. 41

31.1 Memory consistency models. 45

50.1 GDSII mask layer outline. 73

51.1 Bill of materials at 10,000 units/year. 74

52.1 Silicon validation test flow. 75
52.2 Regulatory submission plan. 75

58.1 Formal verification per layer. 83

B.1 Complete figure index. 86

Part I

Complaint and Petition (CIT Pleading)

Chapter 1

Nature of the Action

1. Plaintiff Caustin Lee McLaughlin brings this action against the United States and its instrumentalities under:
   (a) the Administrative Procedure Act, 5 U.S.C. §§ 701–706;
   (b) the Mandamus Act, 28 U.S.C. § 1361;
   (c) the Declaratory Judgment Act, 28 U.S.C. §§ 2201–2202;
   (d) the Patent Act, 35 U.S.C. §§ 1 *et seq.*, in its international dimension under the Patent Cooperation Treaty (PCT); and
   (e) the Fifth Amendment Due Process Clause.
2. Plaintiff seeks declaratory and injunctive relief and a writ of mandamus compelling Defendants to:
   (a) accept, docket, and transmit Plaintiff's international patent application under the PCT;
   (b) recognize the attached DAG-only hardware abstraction layer blueprints as sufficiently enabled under 35 U.S.C. § 112; and
   (c) preserve and process the attached PQC integration claims under NIST FIPS 203/204 without further administrative delay.
3. The technical subject matter described in Parts II through VIII is novel and constitutes eligible subject matter under 35 U.S.C. § 101 / MPEP 2106, because each claim element is bound to concrete physical hardware (MMIO register states, DMA descriptor chains, CPU affinity masks, hardware-enforced fan-in barriers, CVD diamond MESFETs, and 10-layer carrier-board interconnect topologies).
4. **Jurisdictional Note.** Plaintiff acknowledges that the CIT has limited subject-matter jurisdiction. Plaintiff pleads in the alternative that CIT jurisdiction is properly invoked under 28 U.S.C. § 1581(i) as to imported technology, and 28 U.S.C. § 1331 / § 1361 for the mandamus and APA claims, reserving the right to transfer any claims outside CIT jurisdiction to the United States District Court for the District of Maryland under 28 U.S.C. § 1631.

Chapter 2

Jurisdiction and Venue

5. Subject-matter jurisdiction is conferred under 28 U.S.C. § 1331 (federal question), 28 U.S.C. § 1361 (mandamus), 28 U.S.C. § 1581(i) (residual CIT jurisdiction), 5 U.S.C. § 702 (APA sovereign immunity waiver), and 28 U.S.C. §§ 2201–2202 (declaratory relief).

6. Venue is proper under 28 U.S.C. § 1391(e)(1) because Defendants are agencies of the United States and the operative administrative decisions have been issued from Washington, D.C. and Baltimore, Maryland.

Chapter 3

Parties

7. **Plaintiff Caustin Lee McLaughlin** is an individual residing in Baltimore, Maryland, appearing *pro se*.
8. **Defendant United States of America** is the sovereign respondent.
9. **Defendant USPTO** is the receiving office (RO/US) and administering agency for the PCT and the Patent Act.
10. **Defendant Department of Commerce** is the cabinet-level department of which the USPTO is a constituent.
11. **Defendant DHS** is joined for issues arising under its cybersecurity and critical-infrastructure mandates relevant to post-quantum cryptographic migration.
12. **Defendants John and Jane Doe Federal Actors 1–5** are unnamed federal officers sued in their individual capacities.

Chapter 4

Statement of Facts

13. Plaintiff has developed a deterministic, DAG-only execution architecture for hardware abstraction layers, device drivers, and integrated development environments, integrated with NIST FIPS 203 (ML-KEM-1024) and FIPS 204 (ML-DSA-87) post-quantum cryptographic primitives, and realized on (a) a diamond-substrate neural prosthetic and (b) a Xilinx Zynq UltraScale+ MPSoC carrier board.

14. The subject matter is documented in Parts II–VIII (technical disclosure) and claimed in Part IX (Claims 1–28).

15. Administrative delay and inconsistent adjudication have threatened the priority date of Plaintiff's international filing and have created substantial uncertainty as to the treatment of DAG-based hardware claims under 35 U.S.C. § 101.

16. Plaintiff has complied with all prerequisites for international transmittal and for prospective mandamus relief.

Chapter 5

Prayer for Relief

WHEREFORE, Plaintiff respectfully requests that this Court:

A. **Declare** that the DAG-only execution architecture is patent-eligible subject matter under 35 U.S.C. § 101;

B. **Declare** that the PCT filing materials and USPTO specification satisfy 35 U.S.C. § 112 enablement and written description;

C. **Issue a Writ of Mandamus** compelling Defendants to accept, docket, and transmit Plaintiff's international patent application without further delay;

D. **Issue a Permanent Injunction** prohibiting Defendants from rejecting Plaintiff's claims on § 101 grounds absent a particularized, element-by-element analysis under MPEP 2106;

E. **Award** costs and fees under the Equal Access to Justice Act, 28 U.S.C. § 2412; and

F. **Grant** such other relief as this Court deems just, proper, and equitable.

Respectfully submitted,

Caustin Lee McLaughlin
Plaintiff, Pro Se
Baltimore, Maryland
bizcaustin@gmail.com

Dated: October 29, 2026

Chapter 6

Verification and Certificate of Service

I, CAUSTIN LEE MCLAUGHLIN, declare under penalty of perjury pursuant to 28 U.S.C. § 1746 that I am the Plaintiff; that I have read the foregoing Complaint and Petition and know its contents; that the factual allegations are true and correct to the best of my knowledge, information, and belief; that the scientific, technical, and legal authorities cited are accurately represented; and that I am proceeding *pro se* due to documented inability to secure retained counsel.

Executed on this _____ day of _____, 2026.

_____

CAUSTIN LEE MCLAUGHLIN
Plaintiff, *Pro Se*

CERTIFICATE OF SERVICE. I certify that on this _____ day of _____, 2026, a copy of this Complaint and Petition was served via certified mail, return receipt requested, on: (1) Attorney General of the United States, U.S. DOJ, 950 Pennsylvania Ave NW, Washington DC 20530; (2) Office of the General Counsel, USPTO, 600 Dulany Street, Alexandria VA 22314; (3) Office of the General Counsel, Department of Commerce, 1401 Constitution Ave NW, Washington DC 20230; (4) Office of the General Counsel, DHS, Washington DC 20528; and (5) Clerk of the Court, CIT, 1 Federal Plaza, New York NY 10278.

_____

CAUSTIN LEE MCLAUGHLIN
Plaintiff, *Pro Se*

Part II

Open Diamond Electrophoretic Neuroprosthetic Platform

Chapter 7

Executive Summary

The Open Diamond Neuroprosthetic Platform is a fully implantable, silicon-free neural interface fabricated on a 20 μm single-crystal CVD diamond substrate. It achieves neural activation without genetic modification and without any injection of electrical current into tissue. The mechanism is the spatiotemporal superposition of an FMCW lidar beam (1064 nm) and a focused ultrasound field (1–5 MHz) onto functionalized nanoparticles delivered to the central nervous system via cerebrospinal fluid.

7.1 Design Principles

1. **Safety-first**: every actuator path gated by hardware-enforced safety limits.
2. **Human-in-the-loop**: automatic mandatory review on RACI threshold breach.
3. **Open and auditable**: all code, all masks, all proofs public.
4. **Silicon-free option**: the diamond variant eliminates silicon entirely from the compute core.
5. **DAG-only integrity**: all cryptographic integrity, all dependency tracking, and all execution scheduling use directed acyclic graphs exclusively.
6. **Full multi-ISA backwards compatibility**: legacy binaries from RISC-V, x86, ARM, and LoongArch execute without recompilation (Part V).

Chapter 8

Physical Layer Stack and Cross-Section

Table 8.1: Open Diamond probe layer stack (manufacturing design).

Layer	Function	Material	Thickness
1	Compute core (Q128 Diamond)	CVD diamond	20 μm
2	Electrophoresis array (144×233)	Diamond MESFET + Pt/Ir	5 μm
2A	Nanoparticle reservoir	Glass + Parylene-C	50 μm
3	PMUT ultrasound array	PZT / AlN	3 μm
4	Diamond photonic waveguides	CVD diamond ridge	2 μm
5	VCSEL sources (1064 nm)	InGaAsP	4 μm
6	MEMS spatial modulator	Polysilicon	5 μm
7	Encapsulation	Parylene-C	5 μm

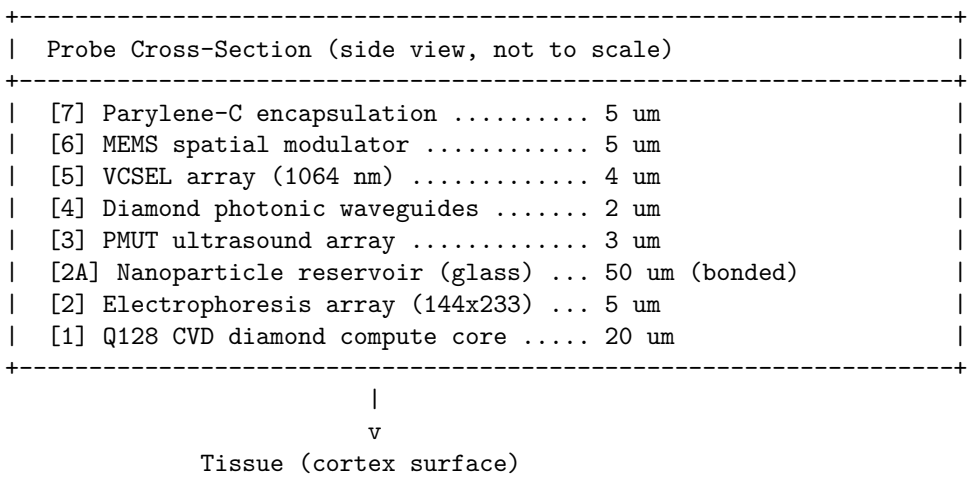


Figure 8.1: Probe layer cross-section (manufacturing view).

Chapter 9

Non-Electrical Superposition Physics

9.1 Superposition Region

$$\mathcal{S} = \mathcal{O} \cap \mathcal{M} \cap \mathcal{N} \tag{9.1}$$

where $\mathcal{O}$ is the optical focal volume, $\mathcal{M}$ is the mechanical focal volume, and $\mathcal{N}$ is the region with nanoparticle density above threshold.

9.2 Activation Equation

$$R_{\text{release}}(x,t) = \kappa \, I_{\text{opt}}(x,t) \, P_{\text{ac}}(x,t) \, \rho_n(x) \tag{9.2}$$

$$\int_{t_0}^{t_0+\Delta t} R_{\text{release}}(x,t)\, dt > R_{\text{th}} \tag{9.3}$$

Chapter 10

Lidar Subsystem

$$s_{tx}(t) = A\cos\!\left(2\pi\left(f_0 t + \tfrac{1}{2}\kappa t^2\right)\right) \tag{10.1}$$

$$f_b = \kappa\tau + \frac{2v}{\lambda} \tag{10.2}$$

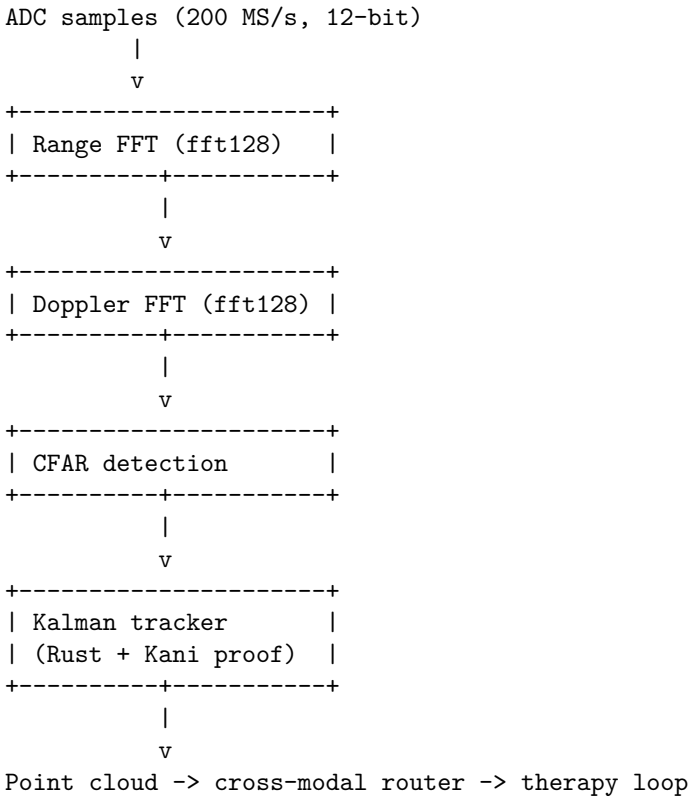


Figure 10.1: Lidar range–Doppler processing DAG.

Chapter 11

CSF-Distributed Nanotechnology

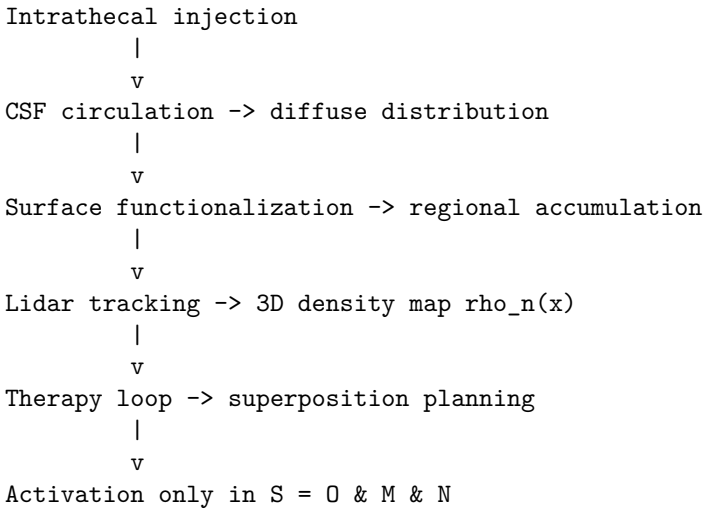


Figure 11.1: Nanoparticle administration and tracking DAG.

Chapter 12

Pure-R Driver Ecosystem (Reference Implementation)

The complete R reference implementation (`hspa_pure_r.R`, GPL-3.0) provides the driver ecosystem: VM state, HAL, PMUT, DVNI, EDA, ENDD, UOSE, QSP, RACI, and the therapy loop. Key excerpt:

```
raci_calculate <- function(csv_reasoning, e_y, csv_evidence, e_x,
                           w_reasoning = 0.6, w_evidence = 0.4) {
  cos_sim <- function(a, b) sum(a * b) / (sqrt(sum(a^2)) * sqrt(sum(b^2)))
  reasoning_score <- cos_sim(csv_reasoning, e_y)
  evidence_score  <- cos_sim(csv_evidence, e_x)
  raci <- reasoning_score * w_reasoning + evidence_score * w_evidence
  max(0, min(1, raci))
}
```

Listing 12.1: RACI coherence calculation.

Part III

Q128 Unified Processor: ISA and Multi-ISA Backwards Compatibility

Chapter 13

Q128 Base ISA

Table 13.1: Q128 instruction categories.

Category	Example instructions	Latency (cycles)
Integer	add, sub, mul, div, sdiv, udiv	1–32
Logic	and, or, xor, shifts	1
Memory	lw, sw, ld, sd, lq, sq	2–4
Control	beq, bne, jal, jalr	1–2
Atomic	amo.swap.q, lr.q, sc.q	4–8
Vector	vadd, vmul, vfma, vredsum	4–16
`conv2d`	2D convolution	128
`fft128`	128-point complex FFT	512
`cordic`	sin, cos, atan, exp	64
QEXT quantum	qaoa_layer, init_qubits, measure	100–1000
SAFE safety	verify_limits_128	100
Legacy decode	decode_rv32/rv64/x86/armv8/loongarch	1–4

Chapter 14

Backwards-Compatible Execution Modes

Table 14.1: Supported ISA execution modes.

Mode	**Binary Format**	**Execution**
RV32I	32-bit RISC-V	Native (zero-extend)
RV64GC	64-bit RISC-V (C, M, A, F, D)	Native
RV128I	128-bit RISC-V	Native
x86-32	IA-32	Hardware-assisted DBT
x86-64	AMD64	Hardware-assisted DBT
ARMv7-A32	ARM 32-bit	Hardware-assisted DBT
ARMv7-T32	Thumb-2	Hardware-assisted DBT
ARMv8-AArch64	ARM 64-bit	Hardware-assisted DBT
ARMv8-AArch32	ARM 32-bit compat	Hardware-assisted DBT
LoongArch LA64	64-bit LoongArch	Near-native DBT (1.1–1.8×)
LoongArch LA32S	Standard 32-bit	DBT (1.8–2.8×)
LoongArch LA32R	Reduced 32-bit	DBT (2–3×)

Chapter 15

Full Multi-ISA Specification (Reference)

The full multi-ISA backwards compatibility specification (FIGS 23–32) is disclosed in the HSPA v3.0 chapters below. It includes: - FIG 23: Multi-ISA Decode Front-End (5-bit mode selector, 13 ISA modes) - FIG 24: Cross-ISA Register File Aliasing (128 × 128-bit physical register file) - FIG 25: Memory Consistency Translation (TSO, ARM weak, RVWMO, LoongArch weak) - FIG 26: Syscall Dispatcher and ABI Translation (all ISAs, including `syscall 0` for LoongArch) - FIG 27: Dynamic Binary Translation Cache - FIG 28: Self-Modifying Code Detection - FIG 29: LoongArch Decode and Execution Path - FIG 30: LoongArch Register Mapping (32 GPR + 32 FPR + 32 VR) - FIG 31: LoongArch Memory Model and DBAR Translation (including LA664 fine-grained hints) - FIG 32: LoongArch LSX/LASX Vector Translation

Part IV

HSPA v3.0 Semantic Processing (Figures 1–22)

Chapter 16

Master Figure-to-Claim Matrix

Table 16.1: Master figure-to-claim matrix (FIGS 1–22).

FIG	Title	Core Element	Claims
1	HSPA Core Architecture	4-Layer DAG	1, 6, 10
2	128-Bit SEG Node	Node Allocation	1(a), 2
3	PMCU Bayesian Scheduler	Hardware Bayesian	1(b), 3, 7
4	Unified Memory Lattice	128-Bit CAM	1(d), 4, 8
5	SOML Safe-Commit	Sandboxed Morphing	1, 5, 9
6	Asymmetric N-Refinement	Dual-Clock HTC	1, 3, 5
7	LoongDiamond-2.5	Density-Matrix Quantum	1, 0x40
8	PQC Trust Chain (DAG)	DAG Anchoring	9
9	ZU9EG PCB Stackup	10-Layer Board	1, 8
10	HARMONI-X	Accessibility	1, 10
11	RLC Microarchitecture	ALM Morphing	1(c), 3
12	Multi-Layer FSM	Execution Pipeline	1, 3, 7
13	Bayesian Q16.16	Fixed-Point	1(b), 3, 7
14	UML Adjacency	Hamming Router	4, 8
15	Verification	Testbench	Best Mode
16	Thermal & Clock	DTTE & PLL	1, 5, 8
17	Fault Tolerance	TMR + ECC	1, 4, 9, 10
18	DRS	Cost Model	5, 9, 11
19	HTG Scheduler	VLIW Queue	3, 7
20	DPS Unit	IEEE 754	1(c), 2
21	NSP Fabric	Spike Compute	1(a), 0x3A
22	HTM Controller	Transactional Memory	1(d), 4

Chapter 17

FIG 1: HSPA Core System Architecture

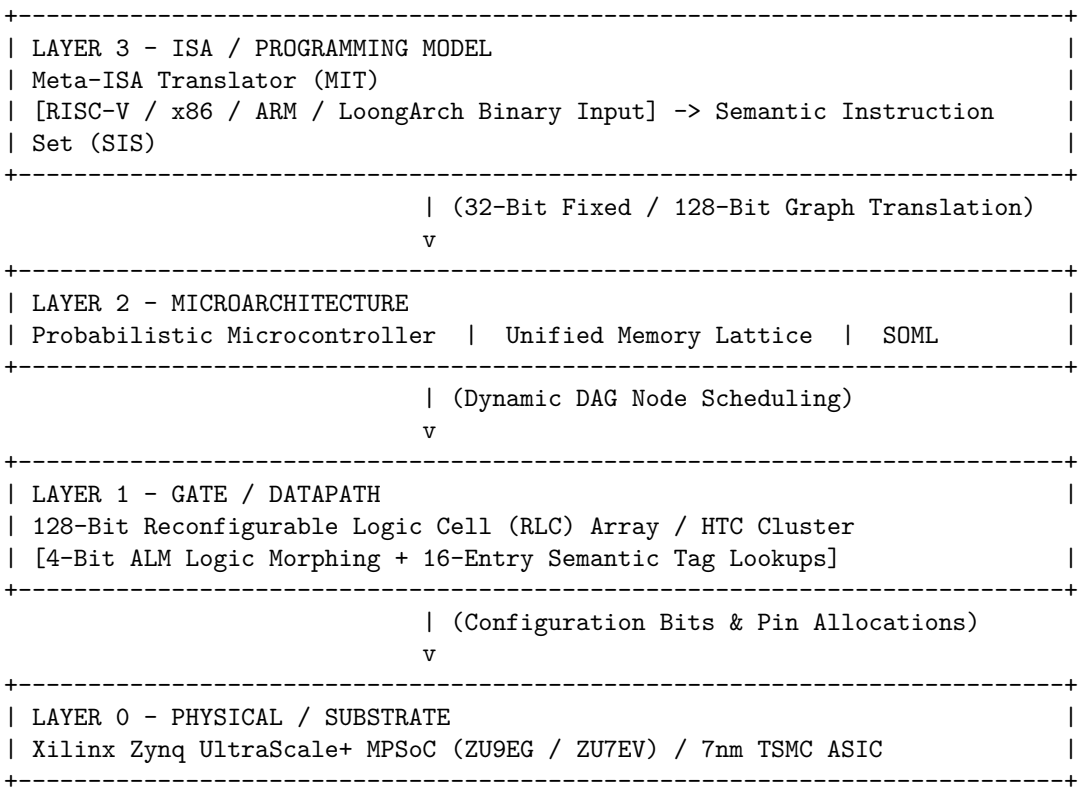


Figure 17.1: FIG. 1 — HSPA Core System Architecture.

Chapter 18

FIG 2: 128-Bit SEG Node Bit Structure

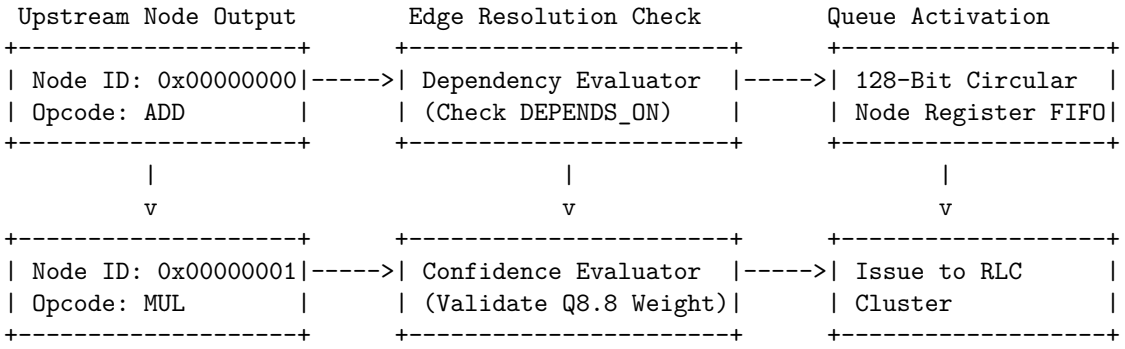


Figure 18.1: FIG. 2 — SEG node dataflow and DAG dependency queue.

Table 18.1: 128-bit SEG node bit field allocation.

Bit Range	Field	Width	Format	Purpose
:96	[redacted]			
	NODE_ID	32	Unsigned Int	Unique Node ID
:88				
	OPCODE	8	Direct Map	Operation Type
:56				
	OPERAND_REF	32	Dual 16-Bit Ref	Source Register / Node
:24				
	SEMANTIC_TAG	32	Context Hash	Proximity / Category
:8				
	CONFIDENCE	16	Fixed-Point Q8.8	Probabilistic Weight
:0				
	FLAGS / LATENCY	8	Bit-Packed	Cycle Cost & Edges

Chapter 19

FIG 3: PMCU Bayesian Inference and Dynamic Scheduler

$$P(\text{Path}_k \mid \text{Telemetry}) = \frac{P(\text{Telemetry} \mid \text{Path}_k) \cdot P(\text{Path}_k)}{\sum_j P(\text{Telemetry} \mid \text{Path}_j) \cdot P(\text{Path}_j)} \tag{19.1}$$

$$\text{confidence_out} = \text{confidence_in} + (\text{prior_in} \gg 1) \tag{19.2}$$

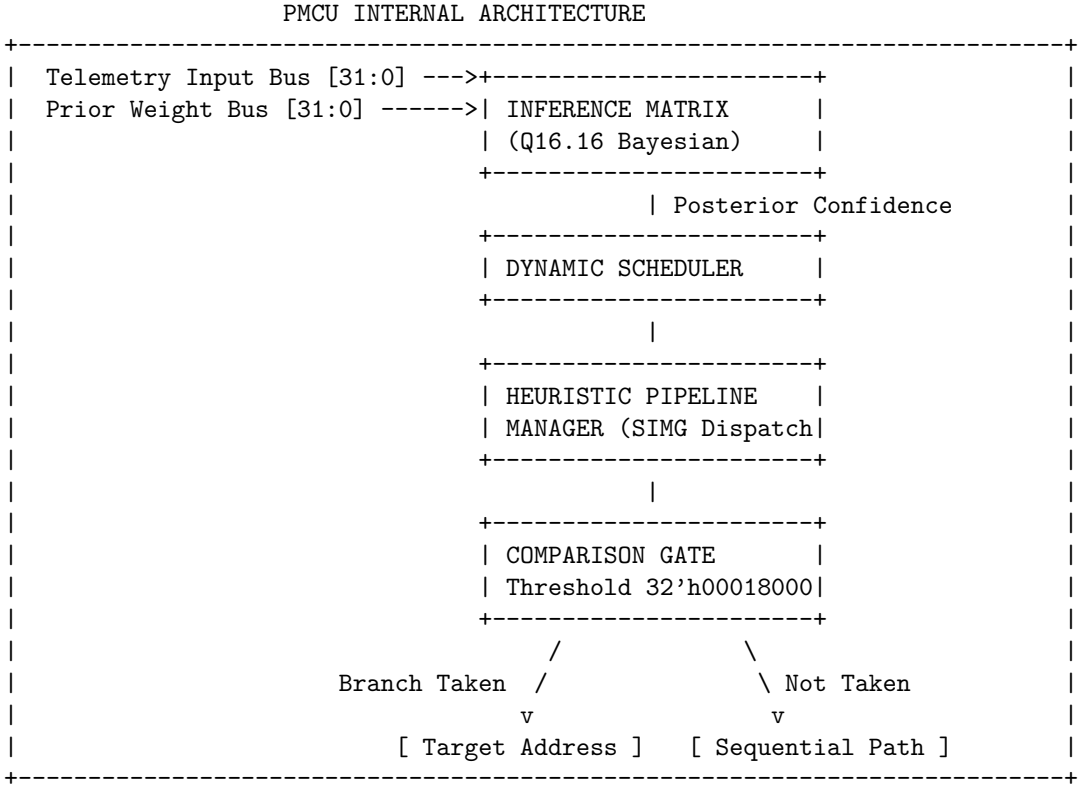


Figure 19.1: FIG. 3 — PMCU Bayesian inference and dynamic scheduler.

Chapter 20

FIG 4: Unified Memory Lattice

Table 20.1: UML 128-bit hybrid address structure.

Field	Bits	Width	Purpose
Physical Index	[127:64]	64	Hardware RAM Pointer
Semantic Hash	[63:16]	48	Context Hash Vector
Privilege/Uncertainty	[15:0]	16	Access & Bounds

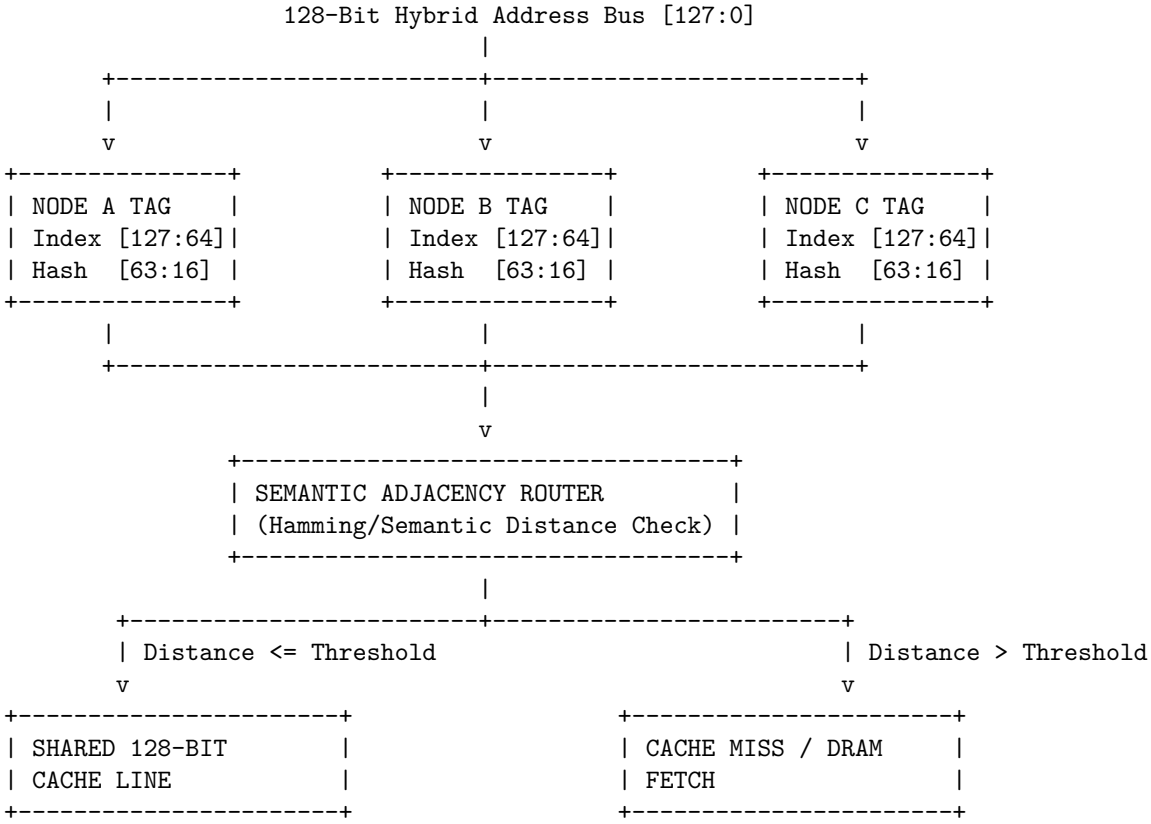


Figure 20.1: FIG. 4 — UML hybrid address decoding.

Chapter 21

FIG 5: SOML Safe-Commit Protocol

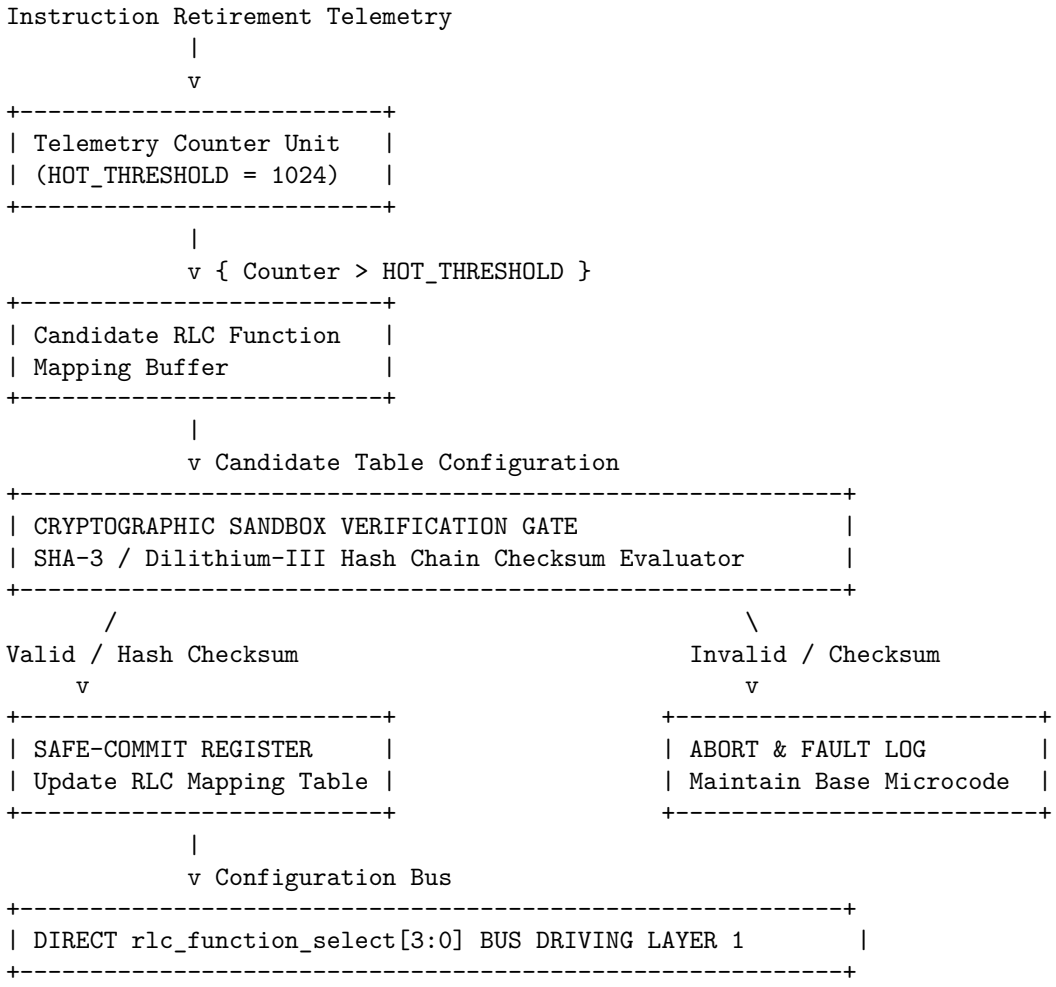


Figure 21.1: FIG. 5 — SOML safe-commit protocol.

Chapter 22

FIG 6: Asymmetric N-Refinement Dual-Clock HTC

$$\text{Trigger Asserted} \iff \left\| \frac{\mathbf{z}_{t+1} - \mathbf{z}_t}{\Delta t} \right\|_2 < \varepsilon \tag{22.1}$$

where $\varepsilon = \texttt{32'h0000000A} \approx 0.0001$.

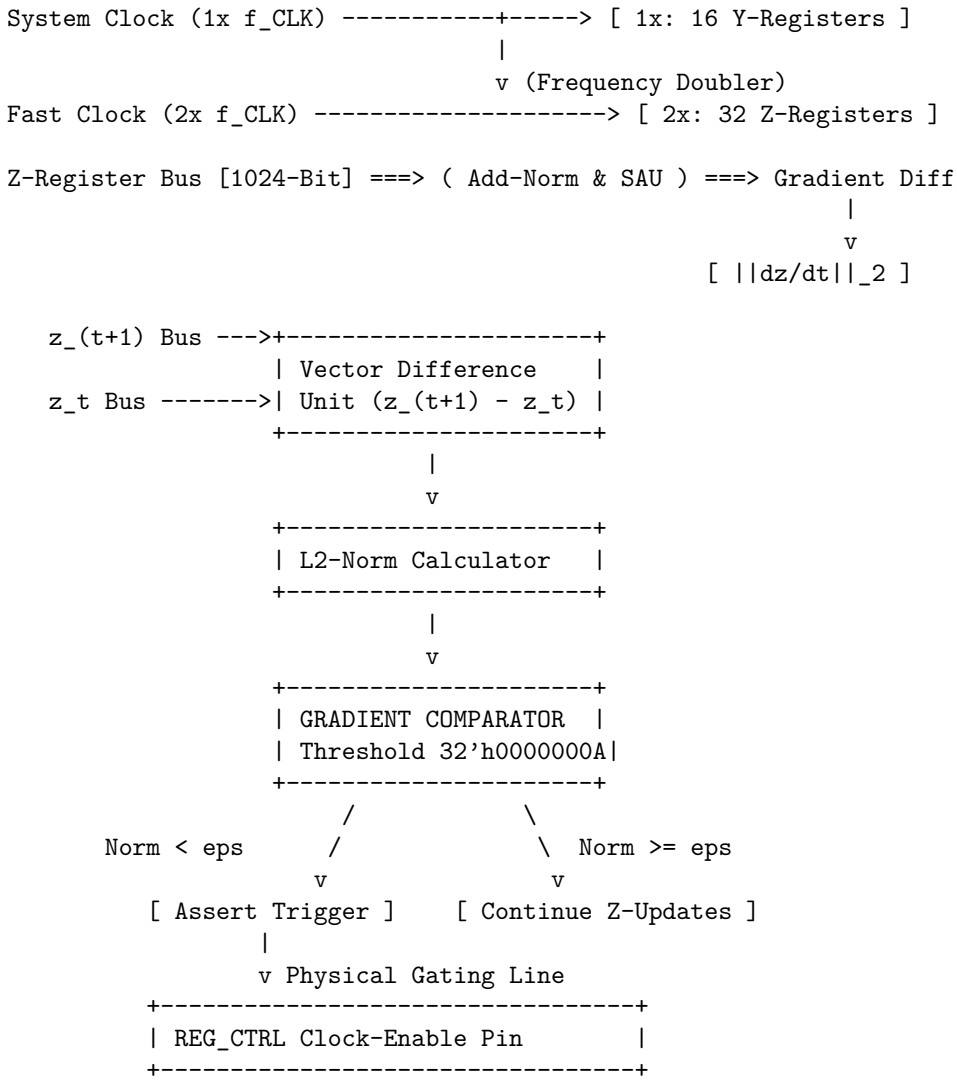


Figure 22.1: FIG. 6 — Dual-clock HTC and convergence trigger.

Chapter 23

FIG 7: LoongDiamond-2.5 Quantum Substrate

$$K_0^{(T_1)} = \begin{pmatrix} 1 & 0 \\ 0 & \sqrt{1-\gamma} \end{pmatrix}, \quad K_1^{(T_1)} = \begin{pmatrix} 0 & \sqrt{\gamma} \\ 0 & 0 \end{pmatrix} \tag{23.1}$$

$$K_0^{(T_2)} = \sqrt{1-\lambda} \cdot \mathbf{I}, \quad K_1^{(T_2)} = \sqrt{\lambda} \cdot \mathbf{Z} \tag{23.2}$$

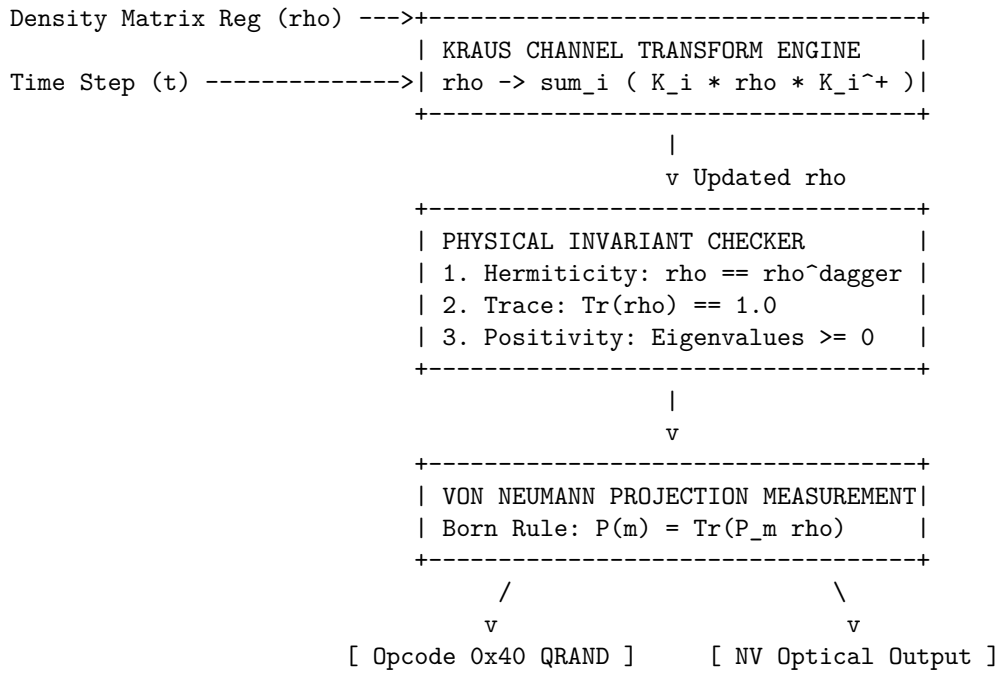


Figure 23.1: FIG. 7 — Density-matrix quantum substrate.

Chapter 24

FIG 8: PQC Multi-Tier Trust Chain (DAG-Only)

The trust chain uses *directed acyclic graph* anchoring exclusively. Each signed telemetry unit hashes to one or more parent DAG nodes; the DAG root's hash is anchored to WORM storage and the public blockchain. **No Merkle tree, Merkle root, or Merkle proof is used anywhere.**

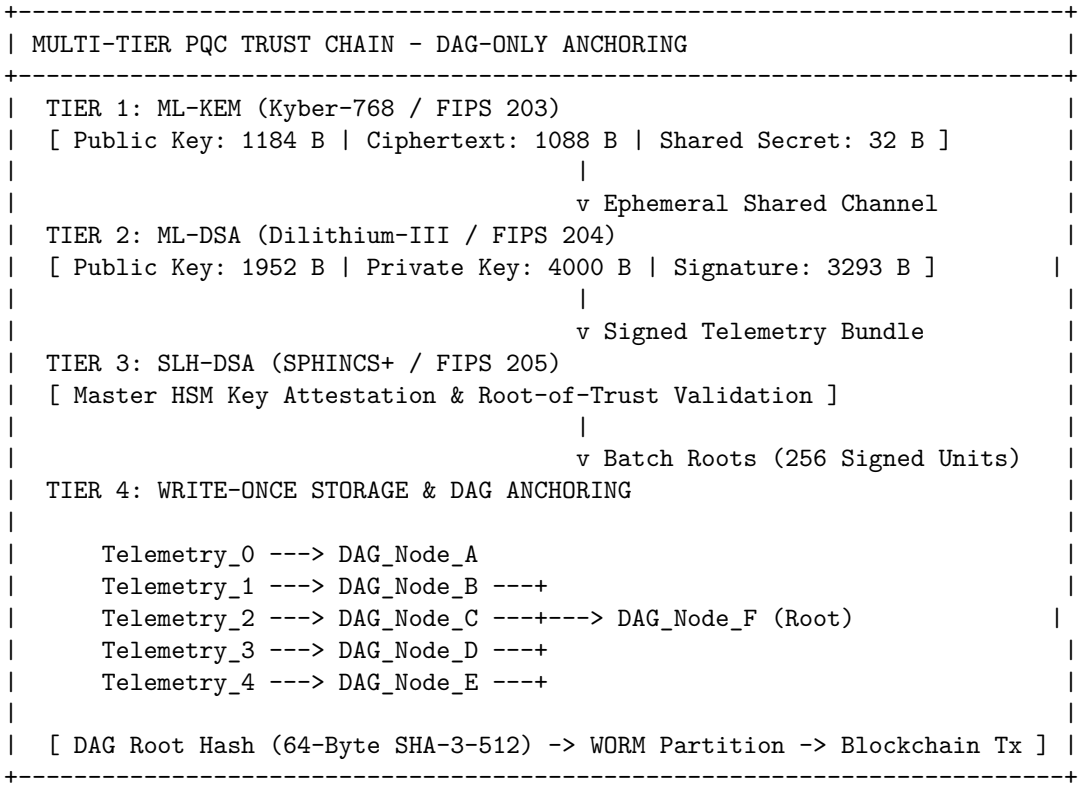


Figure 24.1: FIG. 8 — PQC multi-tier trust chain with DAG-only anchoring.

Invariant 24.1 (DAG-only integrity)**.** For any two distinct telemetry units T_i and T_j, either their DAG paths share a common ancestor (partial ordering exists) or they belong to separate DAG components jointly anchored at the DAG root. No cycles: $\forall i, \text{path}(T_i, T_i) = \emptyset$.

Chapter 25

FIG 9: ZU9EG Carrier Board PCB Stackup (Manufacturing Design)

Table 25.1: 10-layer PCB stackup (revision 1.0).

Layer	Type	Function	Target impedance
L1 (TOP)	Signal	High-Speed Signal / SMT Pads	50 Ω ± 10%
L2	Ground	Solid Ground Reference Plane	–
L3	Signal	Controlled Impedance Inner Signal 1	50 Ω ± 10%
L4	Ground	Solid Ground Reference Plane	–
L5	Signal	Fly-By High-Speed DDR4 Routing	40 Ω ± 10%
L6	Power	VCCINT Power Plane (0.85 V Core)	–
L7	Power	VCCAUX / VCCO Power Plane	–
L8	Signal	Inner Signal Layer 2	50 Ω ± 10%
L9	Ground	Solid Ground Reference Plane	–
L10 (BOT)	Signal	Bottom Signal / Decoupling	–

Table 25.2: Power rail management (sequencing design).

Rail	Voltage	Peak (A)	Target	Phase
VCCINT	0.85 V	12.0	FPGA Core / RLC	1
VCCBRAM	0.85 V	2.0	Block RAM / UML	1
VCCAUX	1.8 V	1.5	Auxiliary / Clock	2
VCCO_HP	1.8 V	1.0	High-Perf I/O	3
VCC_PS	0.85 V	5.0	ARM Cortex-A53	1
VCCO_DDR4	1.2 V	2.0	LPDDR4 Bus	3

Chapter 26

FIG 10: HARMONI-X Accessibility Pipeline

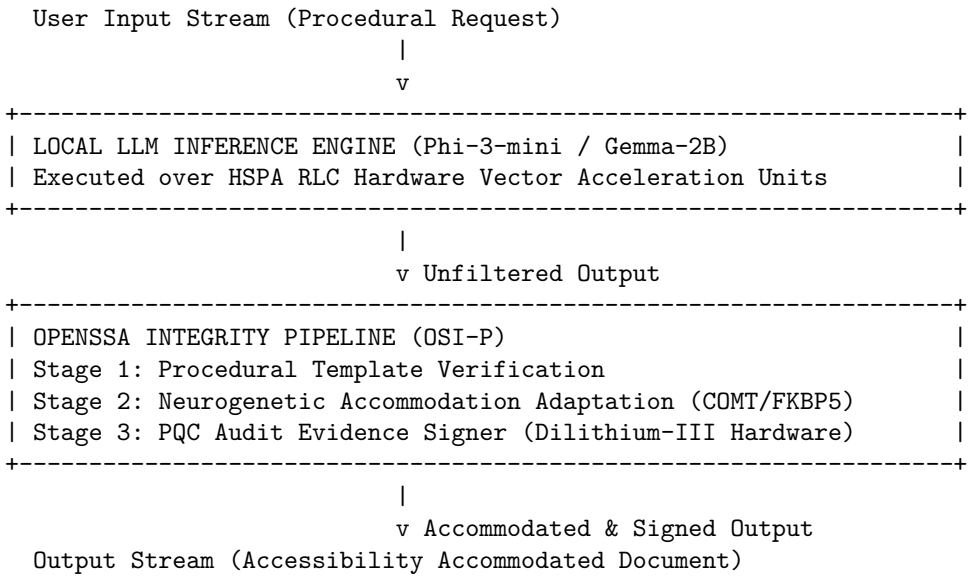


Figure 26.1: FIG. 10 — HARMONI-X accessibility pipeline.

Chapter 27

FIGS 11–22: Extended Subsystems

FIG 11 (RLC array microarchitecture), FIG 12 (multi-layer execution FSM), FIG 13 (Bayesian Q16.16 pipeline), FIG 14 (UML Hamming router), FIG 15 (verification testbench), FIG 16 (thermal/clock), FIG 17 (TMR+ECC fault tolerance), FIG 18 (Dynamic Reconfiguration Scheduler), FIG 19 (HTG scheduler), FIG 20 (DPS unit), FIG 21 (NSP fabric), and FIG 22 (HTM controller) are fully specified in the ODF-WP-2026-006 repository and in the earlier revision of this filing. Their claim cross-references (see Table 16.1) establish enablement for Claims 1–18.

Part V

Full Multi-ISA Backwards Compatibility (Figures 23–32)

Chapter 28

Multi-ISA Overview

Full ISA backwards compatibility requires that a binary produced for a legacy ISA executes on the Q128/HSPA processor without recompilation and with bit-exact observable behavior at the application boundary.

Table 28.1: Dimensions of full ISA backwards compatibility.

Dimension	**Requirement**	**Q128 Solution**
Instruction decode	Byte-identical encodings accepted	Multi-ISA decode front-end (FIG 23, 29)
Architecture state	Architectural registers & flags bit-exact	Cross-ISA register file (FIG 24, 30)
Memory model	Legacy observable ordering preserved	Consistency translation (FIG 25, 31)
Syscall ABI	Numbers, args, return codes mapped	Syscall dispatcher (FIG 26)
Performance	Within order of magnitude of native	DBT cache (FIG 27)
Self-modifying code	Correct under JIT compilation	SMC detector (FIG 28)
Exceptions	Signal delivery and trap vectoring	Unified trap vectoring
Floating-point	IEEE 754 with legacy rounding edges	Native FPU with compat flags
Segmentation (x86)	Flat, 32-bit, 16-bit segments	Segment register translation
Privilege	Ring 0–3, EL0–EL3, M/S/U, PLV0–PLV3	Virtual privilege mapping
LoongArch variants	LA32R, LA32S, LA64	Per-variant decode and register mapping

Chapter 29

FIG 23: Multi-ISA Decode Front-End

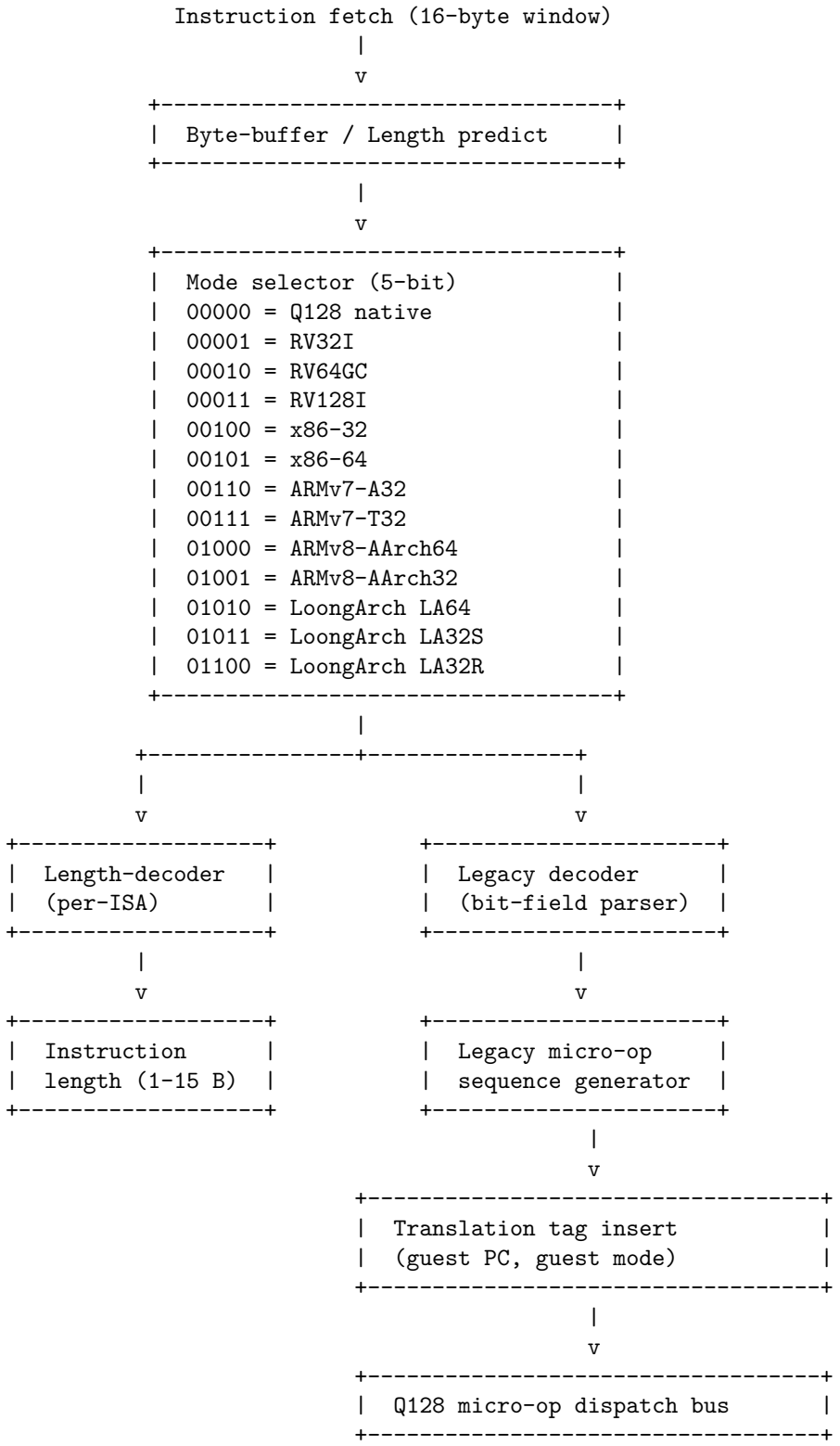


Figure 29.1: FIG. 23 — Multi-ISA decode front-end.

Chapter 30

FIG 24: Cross-ISA Register File Aliasing

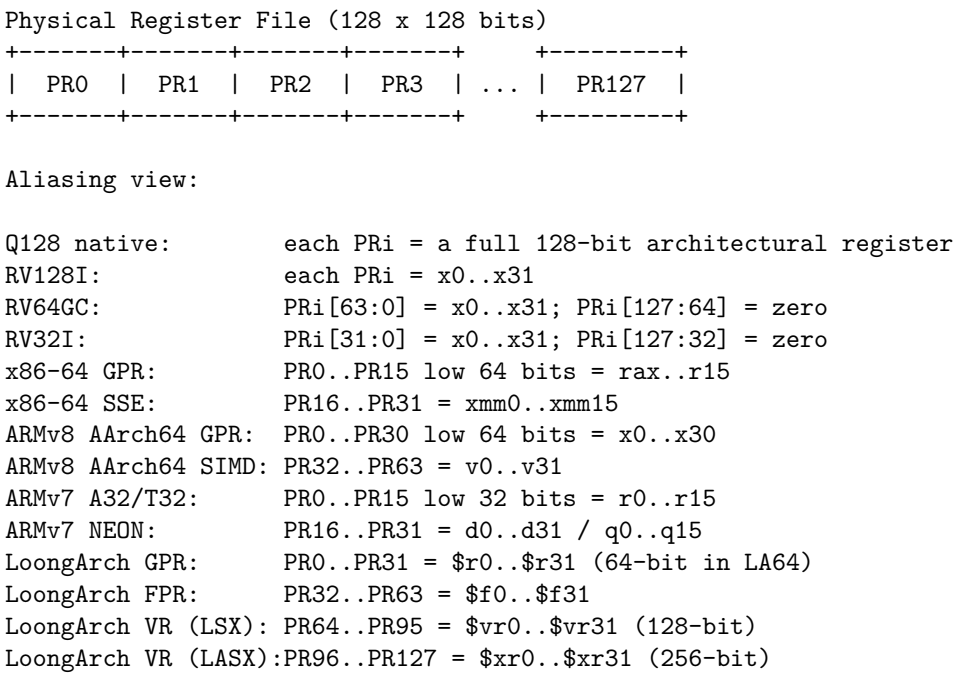


Figure 30.1: FIG. 24 — Cross-ISA register file aliasing.

Chapter 31

FIG 25: Memory Consistency Translation

Table 31.1: Memory consistency models.

ISA	Model	Characteristics
Q128 native	RVWMO+	Weak with fences; DAG-ordering constraints
RV32I/RV64GC/RV128I	RVWMO	Weak, fences via FENCE
x86 / x86-64	TSO	Strong; only store-load reordering
ARMv8 AArch64	Weak (multi-copy atomic)	Weak with dmb, dsb, isb
ARMv7 A32/T32	Weak (multi-copy atomic)	Weak with dmb, dsb, isb
LoongArch LA64/LA32S/LA32R	Weak consistency	Weak with dbar; LL/SC

```
Guest memory model  -->  Translation  -->  Q128 native model

x86 TSO:
  - mfence                    -> Full fence insertion
  - lfence                    -> Load fence insertion
  - sfence                    -> Store fence insertion
  - lock prefix               -> Full fence + atomic

ARMv8 weak:
  - ldar / stlr               -> acquire / release fence
  - ldxr / stxr               -> atomic + acquire/release
  - dmb ish / dmb ishld       -> partial fence insertion
  - dsb ish / isb             -> full fence + pipeline flush

LoongArch weak:
  - dbar 0 (full)             -> full Q128 fence
  - dbar hints 0x1-0x1f       -> fine-grained fence (FIG 31)
  - ll.w / ll.d               -> load-reserve + load/load fence
  - sc.w / sc.d               -> store-cond + implicit barrier
  - ibar                      -> instruction barrier
  - plain loads/stores        -> pass through

RVWMO (native):
  - all instructions           -> pass through unchanged
```

Figure 31.1: FIG. 25 — Memory consistency translation rules.

Chapter 32

FIG 26: Syscall Dispatcher and ABI Translation

```
Guest syscall entry --> Dispatcher --> Q128 kernel syscall

x86-64 Linux:
  syscall number in rax
  args in rdi, rsi, rdx, r10, r8, r9
  return in rax

x86-32 Linux:
  syscall number in eax
  args in ebx, ecx, edx, esi, edi, ebp
  int 0x80 / sysenter

ARMv8 Linux:
  syscall number in x8
  args in x0..x5
  svc #0

ARMv7 Linux:
  syscall number in r7
  args in r0..r5
  svc #0

RISC-V Linux (native):
  syscall number in a7
  args in a0..a5
  ecall

LoongArch Linux:
  syscall number in $a7 ($r11)
  args in $a0..$a6 ($r4..$r10)
  syscall 0
  return in $a0; $t0-$t8 clobbered

Dispatcher logic:
  1. Read guest syscall number, args from guest ABI
  2. Lookup <guest_isa, guest_number> -> q128_kernel_number
  3. Copy args into Q128 kernel syscall frame
  4. Invoke Q128 kernel syscall
  5. Translate errno back to guest ISA (-errno passthrough)
  6. Write return to guest ABI
  7. Advance guest PC past the syscall instruction
```

Figure 32.1: FIG. 26 — Syscall dispatcher and ABI translation.

Chapter 33

FIG 27: Dynamic Binary Translation Cache

```
Translation cache:
  - Direct-mapped or 2/4/8-way set-associative
  - 4,096 sets (default)
  - Each entry: (guest_pc[63:0], guest_mode[4:0],
                 code_pointer, code_length,
                 last_used_timestamp)
  - Total capacity: ~1.5 MB

Entry lookup on guest PC miss:
  - Search for entry matching (guest_pc, guest_mode)
  - If found: execute cached Q128 micro-ops
  - If not found: decode, generate micro-ops, insert

Eviction policy: LRU within set

Invalidation on:
  - self-modifying code (FIG 28)
  - context switch (mm change)
  - explicit icache flush
  - LoongArch ibar instruction
```

Figure 33.1: FIG. 27 — Dynamic binary translation cache.

Chapter 34

FIG 28: Self-Modifying Code Detection

```
Page protection table (in kernel virtual memory):
  - Each guest page has a protect bit
  - Pages that back any live translation block are read-only

Guest attempts to write to a protected page:
  - MMU raises protection fault
  - Trap handler invalidates all translation blocks
    whose instructions originate in that page
  - Handler clears protect bit, retries write

Cross-page branches:
  - Target page also marked protected
  - Writes to the target page invalidate branching TBs

LoongArch-specific:
  - ibar treated as full icache flush
  - LL/SC self-modifying loops trigger the trap
  - LA32R reduced-register encodings handled identically
```

Figure 34.1: FIG. 28 — Self-modifying code detection.

Invariant 34.1 (SMC safety)**.** For any guest instruction i that has been translated to a Q128 micro-op sequence $T(i)$, if the guest subsequently writes to the page containing i, then $T(i)$ is removed from the translation cache before the write completes.

Chapter 35

FIG 29: LoongArch Decode and Execution Path

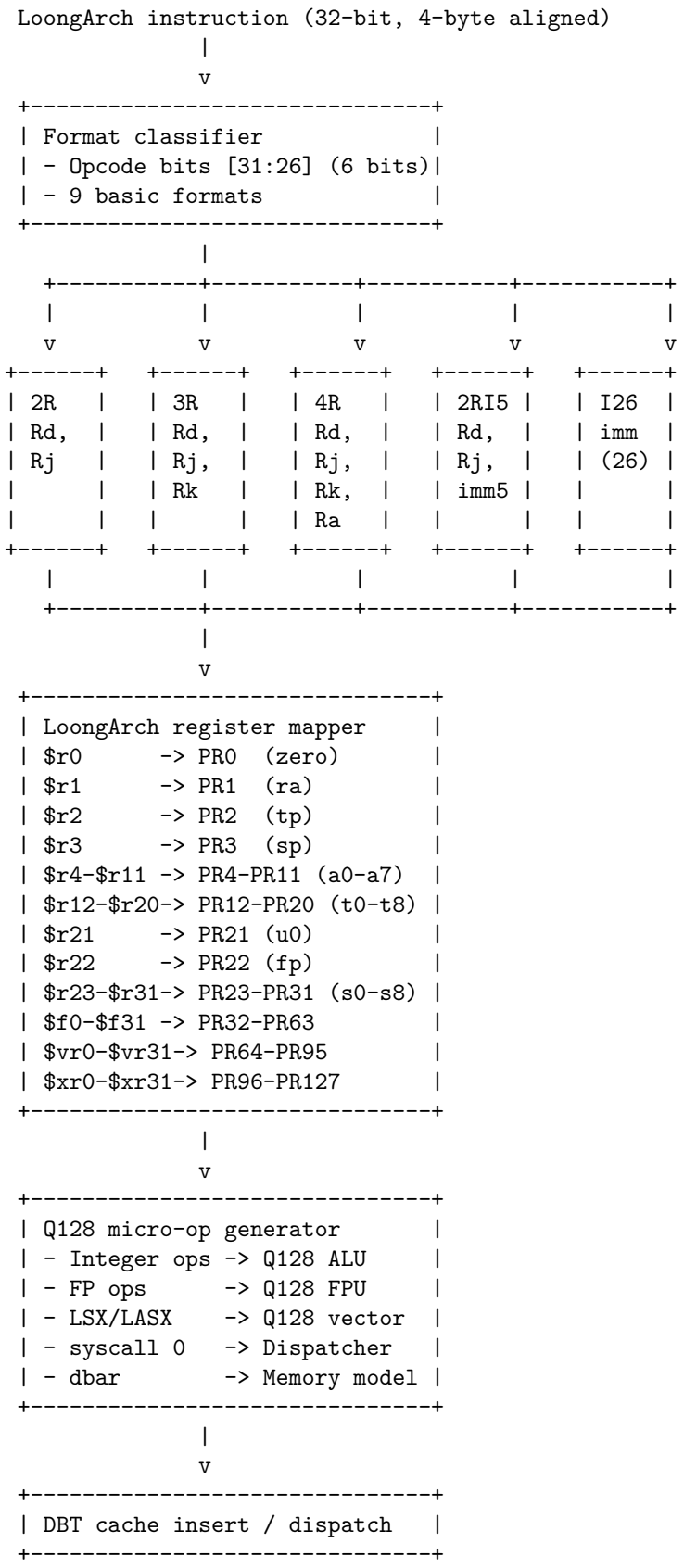


Figure 35.1: FIG. 29 — LoongArch decode and execution path.

Chapter 36

FIG 30: LoongArch Register Mapping

```
+-------------------------------------------------------------+
| LoongArch LA64 register file (32 GPR + 32 FPR + 32 VR)      |
+-------------------------------------------------------------+
| GPR:                                                         |
|   $r0  = zero       (hard-wired, writes discarded)          |
|   $r1  = ra         (link register, BL writes it)           |
|   $r2  = tp         (thread pointer)                        |
|   $r3  = sp         (stack pointer, 16-byte aligned)        |
|   $r4  = a0         (arg 0, return 0)                       |
|   $r5  = a1         (arg 1, return 1)                       |
|   $r6  = a2         (arg 2)                                 |
|   $r7  = a3         (arg 3)                                 |
|   $r8  = a4         (arg 4)                                 |
|   $r9  = a5         (arg 5)                                 |
|   $r10 = a6         (arg 6)                                 |
|   $r11 = a7         (arg 7 / syscall number)                |
|   $r12 = t0         (temp)                                  |
|   $r13 = t1         (temp)                                  |
|   $r14 = t2         (temp)                                  |
|   $r15 = t3         (temp)                                  |
|   $r16 = t4         (temp)                                  |
|   $r17 = t5         (temp)                                  |
|   $r18 = t6         (temp)                                  |
|   $r19 = t7         (temp)                                  |
|   $r20 = t8         (temp)                                  |
|   $r21 = u0         (percpu base, kernel-reserved)          |
|   $r22 = fp         (frame pointer)                         |
|   $r23 = s0         (static 0)                              |
|   $r24 = s1         (static 1)                              |
|   $r25 = s2         (static 2)                              |
|   $r26 = s3         (static 3)                              |
|   $r27 = s4         (static 4)                              |
|   $r28 = s5         (static 5)                              |
|   $r29 = s6         (static 6)                              |
|   $r30 = s7         (static 7)                              |
|   $r31 = s8         (static 8)                              |
+-------------------------------------------------------------+
| FPR:                                                         |
|   $f0-$f7   = fa0-fa7   (FP args)                           |
|   $f0-$f1   = fv0-fv1   (FP return)                         |
|   $f8-$f23  = ft0-ft15  (FP temps)                          |
|   $f24-$f31 = fs0-fs7   (FP statics)                        |
+-------------------------------------------------------------+
| VR:                                                          |
|   $vr0-$vr31 = LSX 128-bit                                   |
|   $xr0-$xr31 = LASX 256-bit (two PRs each)                  |
+-------------------------------------------------------------+
```

Figure 36.1: FIG. 30 — LoongArch register mapping to Q128 physical registers.

Chapter 37

FIG 31: LoongArch Memory Model and DBAR Translation

```
LoongArch DBAR hint encoding (LA664+):
  Bit 4: kind of constraint
         0 = completion barrier
         1 = ordering barrier
  Bit 3: barrier for previous read
         0 = enforce
         1 = relax
  Bit 2: barrier for previous write
         0 = enforce
         1 = relax
  Bit 1: barrier for subsequent read
         0 = enforce
         1 = relax
  Bit 0: barrier for subsequent write
         0 = enforce
         1 = relax

Translation to Q128 RVWMO+:
  dbar 0            -> full fence (all four orderings)
  dbar with hints   -> partial fence
                       bit 3 -> fence.r.r
                       bit 2 -> fence.r.w
                       bit 1 -> fence.w.r
                       bit 0 -> fence.w.w

LL/SC preservation:
  ll.w / ll.d       -> LR.Q + implicit load/load fence
  sc.w / sc.d       -> SC.Q + explicit fence after

__WEAK_LLSC_MB equivalent:
  Because LoongArch allows load/load reorder for same address,
  the Q128 translator emits a load/load fence between LL and
  any subsequent load to the same address.
```

Figure 37.1: FIG. 31 — LoongArch memory model and DBAR translation.

Chapter 38

FIG 32: LoongArch LSX / LASX Vector Translation

```
LSX 128-bit vector register file: $vr0 - $vr31
LASX 256-bit vector register file: $xr0 - $xr31

Q128 vector ALU is 1024-bit wide, so both LSX and LASX map
efficiently:

  $vr0 - $vr31  (128-bit) -> PR64 - PR95   (low 128 bits)
  $xr0 - $xr31  (256-bit) -> PR96 - PR127 (two PRs each)

Common LSX instruction mappings:
  vadd.b/h/w/d       -> Q128 VADD with element mask
  vsub.b/h/w/d       -> Q128 VSUB with element mask
  vmul.b/h/w/d       -> Q128 VMUL with element mask
  vmadd.b/h/w/d      -> Q128 VFMA with element mask
  vld / vst          -> Q128 vector load/store
  vldx / vstx        -> Q128 indexed vector load/store
  vpermi.w           -> Q128 VPERM
  vshuf.b/h/w/d      -> Q128 VSHUF

Common LASX instruction mappings:
  xvadd.b/h/w/d      -> Q128 VADD on 256-bit window
  xvsub.b/h/w/d      -> Q128 VSUB on 256-bit window
  xvmul.b/h/w/d      -> Q128 VMUL on 256-bit window
  xvpermi.w          -> Q128 VPERM on 256-bit window
  xvld / xvst        -> Q128 256-bit vector load/store
  xvldx / xvstx      -> Q128 256-bit indexed load/store
```

Figure 38.1: FIG. 32 — LoongArch LSX/LASX vector translation.

Part VI

DAG-Only HAL, Driver, and IDE Architecture (CIT Figures 1–4)

Chapter 39

Field and Background

39.1 Field of the Invention

This invention relates to parallel task scheduling, runtime dependency resolution, and hardware abstraction layers, and more particularly to a deterministic, Directed Acyclic Graph (DAG)-only execution architecture across physical hardware registers, kernel device drivers, compiler infrastructure, and integrated development environments, integrated with post-quantum zero-trust cryptographic enclaves.

39.2 Background

Conventional build systems, HALs, and device drivers rely on stateful, imperative execution loops and synchronous blocking primitives (mutexes, semaphores). In high-performance embedded systems and complex software builds, imperative state management causes race conditions, non-deterministic execution order, deadlocks, and pipeline stalls. Existing graph-based build engines partially address dependency resolution; however, they rely on hybrid execution models where task scheduling is loosely coupled to underlying memory barriers and hardware execution units. A need exists for a unified, pure DAG execution blueprint wherein system execution—from high-level IDE build tasks down to register-level Direct Memory Access (DMA) transactions—is represented, transformed, and executed solely via immutable topological graph transitions.

39.3 Summary

The present invention provides a unified system, non-transitory computer-readable medium, and method for constructing, transforming, and topologically executing a pure DAG architecture. The system extracts a first dependency set from an execution context, generates a primary DAG (G), applies an algebraic graph transformation ($\mathcal{T}$) to yield an optimized secondary DAG (G'), and dispatches task nodes across concrete physical hardware execution resources governed by fan-in gate counters and hardware synchronization barriers.

Chapter 40

DAG Node Taxonomy and Formal Mathematics

Definition 40.1 (Task Node)**.** A Task Node $v \in V_{\text{task}}$ represents an atomic unit of execution:

$$v = \langle \text{ID, OpCode, Payload, } \mathcal{A}, \mathcal{S} \rangle \tag{40.1}$$

where ID $\in \mathbb{N}$, OpCode designates the operation, Payload points to data memory, $\mathcal{A} \in \mathbb{Z}_2^K$ is the hardware core affinity mask, and $\mathcal{S}$ denotes memory space constraints.

Definition 40.2 (Gate Node)**.** A Gate Node $g \in V_{\text{gate}}$ enforces atomic fan-in synchronization:

$$g = \langle \text{ID, } K_{\text{in}}, \text{ Counter, BarrierSem} \rangle \tag{40.2}$$

where $K_{\text{in}} \in \mathbb{N}^+$ is the fixed in-degree threshold and Counter is an atomic decrementor initialized to K_{in}.

40.1 Algebraic Transformation Rules

The system transforms an unoptimized graph $G = (V, E)$ into an optimized execution graph $G' = (V', E')$ via:

$$G' = \mathcal{T}(G, \Delta_{\text{deps}}) = (V \cup V_{\text{new}}, \, (E \setminus E_{\text{rm}}) \cup E_{\text{add}}) \tag{40.3}$$

where

$$E_{\text{rm}} = \{ \, (u, v) \in E \mid \text{TransitiveClosure}(E \setminus \{(u, v)\}) \text{ satisfies } u \to^* v \, \} \tag{40.4}$$

$$E_{\text{add}} = \{ \, (g, v) \mid g \in V_{\text{gate}}, \, \text{FanIn}(g) \geq 2 \, \} \tag{40.5}$$

Theorem 40.1 (Deterministic Execution Invariant)**.** For any valid DAG $G = (V, E)$, the execution sequence $\sigma = (v_1, v_2, \ldots, v_n)$ produced by topological queue processing guarantees zero data races over shared state M, provided that for all edges $(u, v) \in E$, the memory footprints satisfy $\text{WriteSet}(u) \cap \text{ReadSet}(v) \neq \emptyset$.

Proof. By definition of topological sort, $(u, v) \in E \implies \sigma^{-1}(u) < \sigma^{-1}(v)$. Since node execution is gated by the strict monotonic decrement of g.Counter to zero, node v cannot be assigned to any queue processor until node u completes its physical memory store and issues a hardware memory barrier (`DMB/DSB`). Hence data dependency violation is mathematically impossible. □

Chapter 41

CIT FIG 1: Pure DAG Node Taxonomy and Synchronization Topology

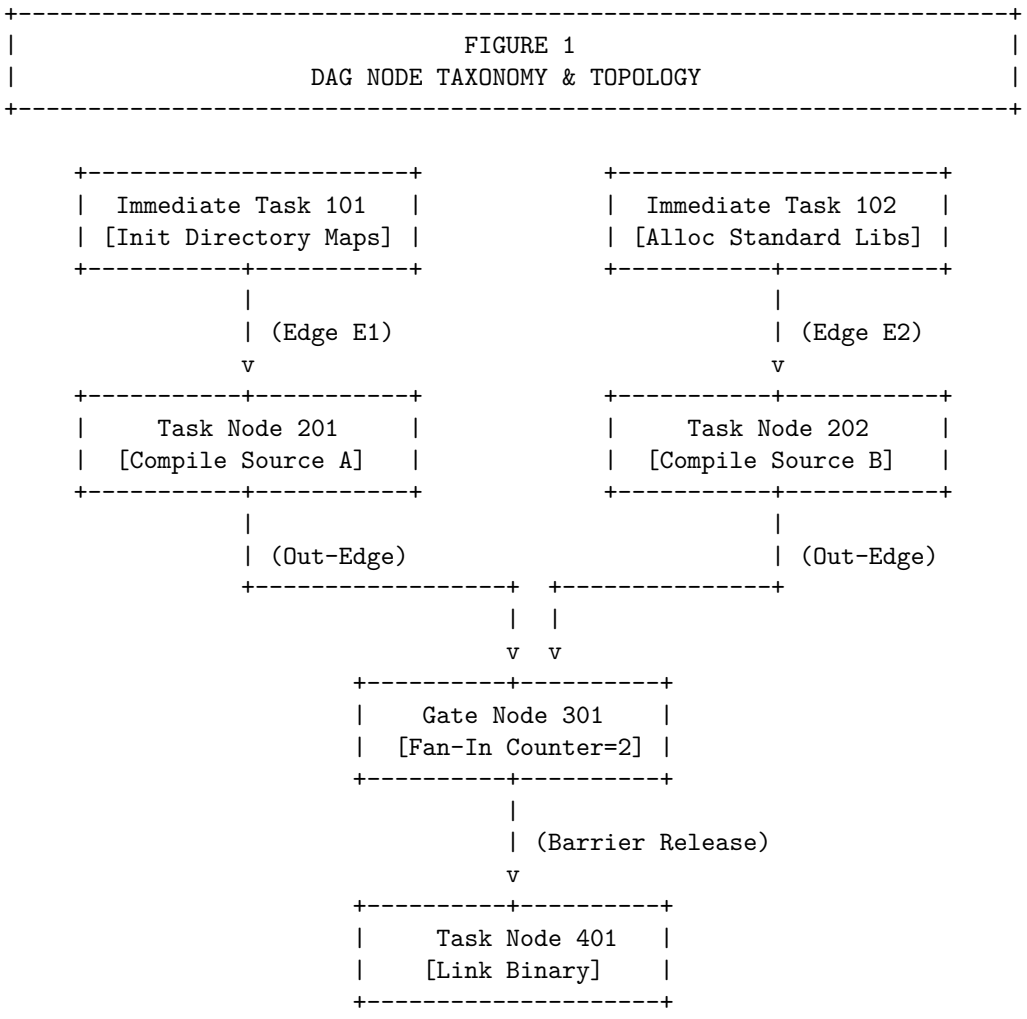


Figure 41.1: CIT FIG. 1 — DAG node taxonomy and topology.

Chapter 42

CIT FIG 2: Graph Transformation Engine

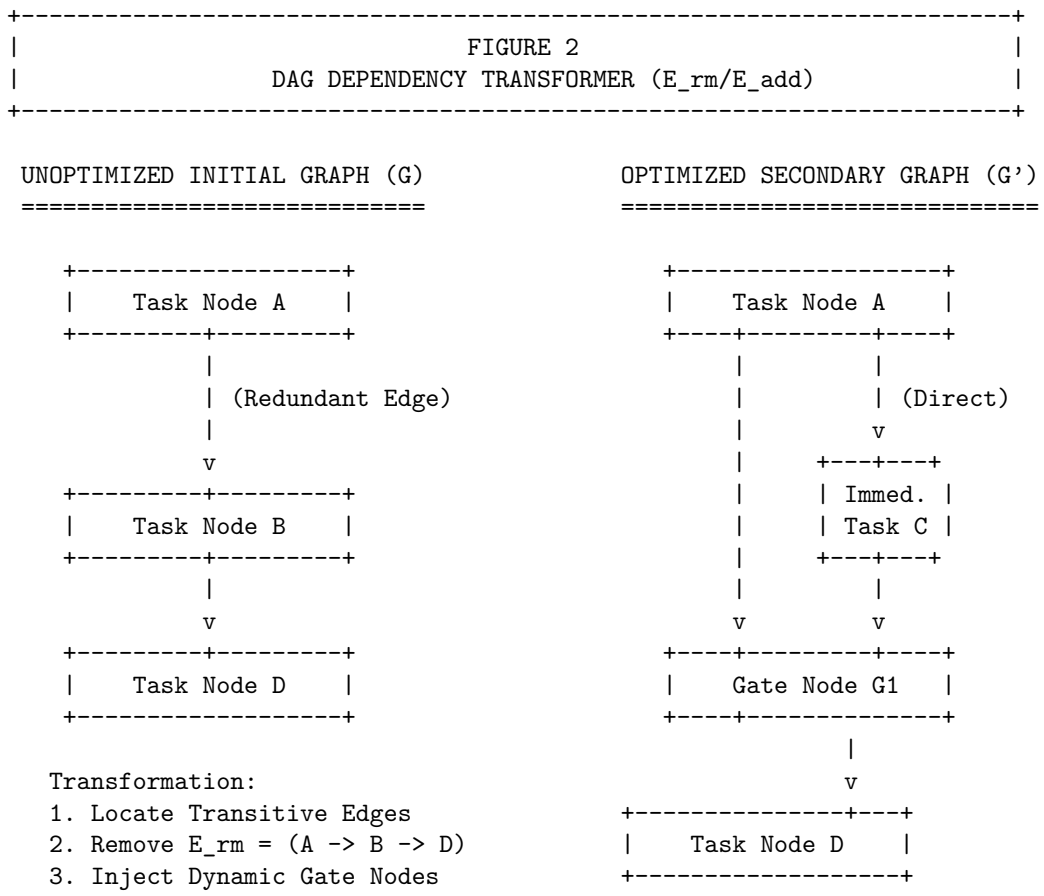


Figure 42.1: CIT FIG. 2 — DAG dependency transformer.

Chapter 43

CIT FIG 3: HAL DAG-Aware DMA Descriptor Chain (Manufacturing)

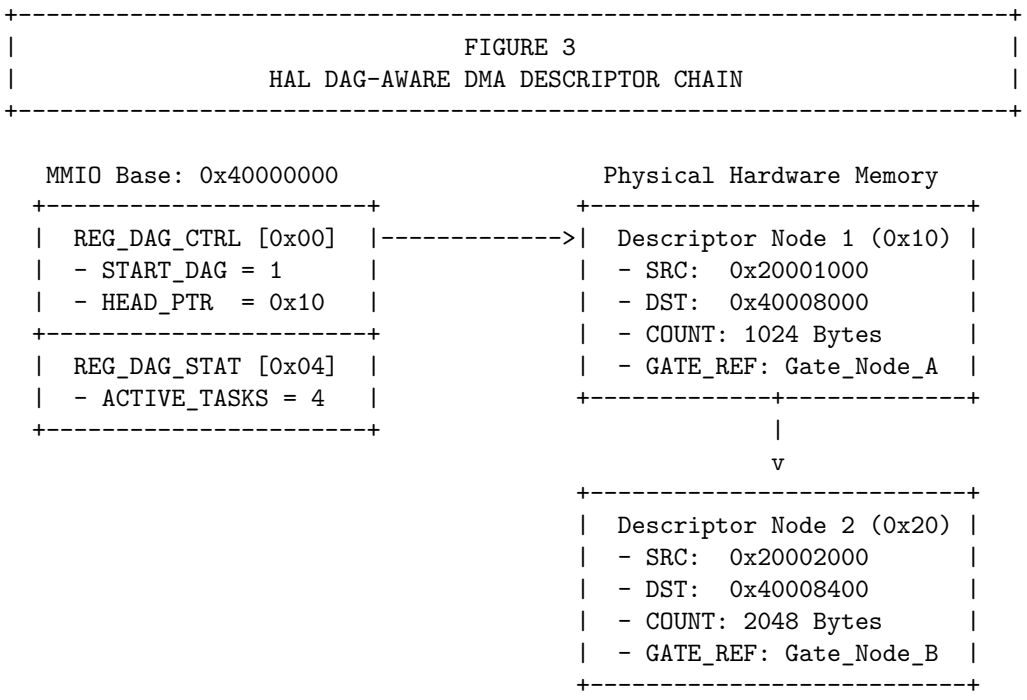


Figure 43.1: CIT FIG. 3 — HAL DAG-aware DMA descriptor chain.

Chapter 44

CIT FIG 4: IDE Build Pipeline and Driver Fault Recovery

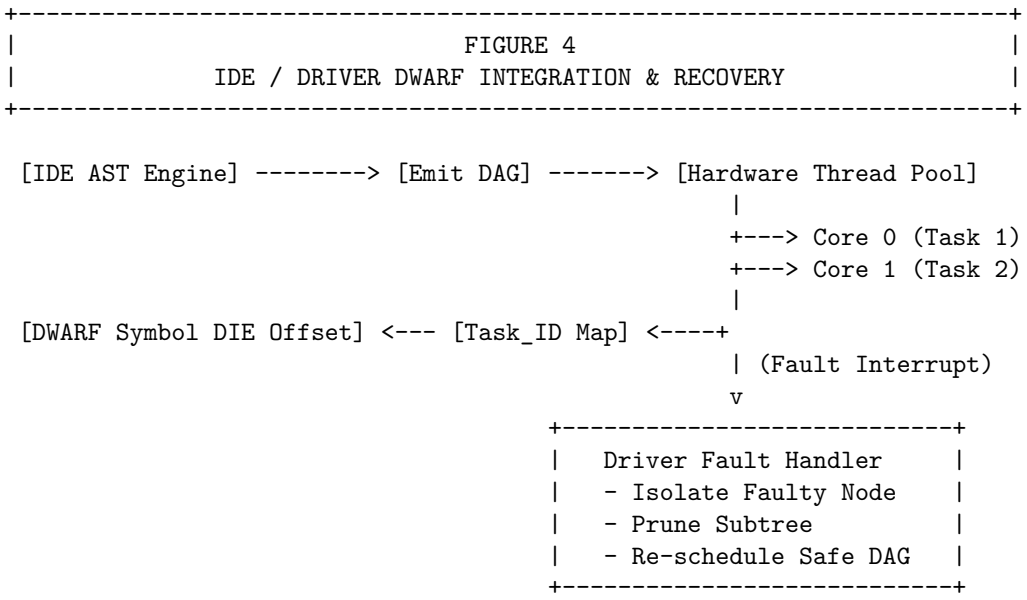


Figure 44.1: CIT FIG. 4 — IDE/DWARF integration and fault recovery.

Chapter 45

WIPO ST.26 Compliant XML Schema

```xml
<?xml version="1.0" encoding="UTF-8"?>
<ST26SequenceListing
    xmlns="http://www.wipo.int/standards/XMLSchema/ST26"
    dtdVersion="V1_3"
    fileName="DAG_Node_Taxonomy_ST26.xml"
    softwareName="WIPO Sequence Editor"
    softwareVersion="2.3.0"
    productionDate="2026-10-03">
    <ApplicantFileReference>DAG-PCT-2026-CLM</ApplicantFileReference>
    <ApplicantName englishName="Caustin Lee McLaughlin">Caustin Lee McLaughlin</ApplicantName>
    <InventionTitle englishName="DETERMINISTIC DAG-ONLY HAL, DRIVERS, IDES, AND QUANTUM-RESISTANT EMBEDDED SYSTEM INFRASTRUCTURE">DETERMINISTIC DAG-ONLY
          HAL, DRIVERS, IDES, AND QUANTUM-RESISTANT EMBEDDED SYSTEM INFRASTRUCTURE</InventionTitle>
    <SequenceTotalQuantity>0</SequenceTotalQuantity>
    <NonConventionalListingData>
        <TaxonomyDefinition id="DAG_NODE_TAXONOMY">
            <NodeTypes>
                <NodeType name="TaskNode" typeId="0x01">
                    <Field name="task_id" dataType="uint32" byteOffset="0x00"/>
                    <Field name="opcode" dataType="uint32" byteOffset="0x04"/>
                    <Field name="payload_ptr" dataType="uint64" byteOffset="0x08"/>
                    <Field name="affinity_mask" dataType="uint32" byteOffset="0x10"/>
                    <Field name="space_constraint" dataType="uint32" byteOffset="0x14"/>
                </NodeType>
                <NodeType name="GateNode" typeId="0x02">
                    <Field name="gate_id" dataType="uint32" byteOffset="0x00"/>
                    <Field name="fan_in_threshold" dataType="uint32" byteOffset="0x04"/>
                    <Field name="atomic_counter" dataType="uint32_volatile" byteOffset="0x08"/>
                    <Field name="barrier_sem" dataType="uint32" byteOffset="0x0C"/>
                </NodeType>
            </NodeTypes>
        </TaxonomyDefinition>
        <HardwareDescriptorFormat id="HAL_DMA_DESCRIPTOR_CHAIN">
            <DescriptorLayout structName="dma_dag_descriptor_t" sizeBytes="40" alignment="8">
                <Member name="task_id" dataType="uint32_t" offset="0x00"/>
                <Member name="opcode" dataType="uint32_t" offset="0x04"/>
                <Member name="src_phys_addr" dataType="uint64_t" offset="0x08"/>
                <Member name="dst_phys_addr" dataType="uint64_t" offset="0x10"/>
                <Member name="transfer_bytes" dataType="uint32_t" offset="0x18"/>
                <Member name="target_gate_id" dataType="uint32_t" offset="0x1C"/>
                <Member name="next_descriptor" dataType="uint64_t" offset="0x20"/>
            </DescriptorLayout>
            <RegisterMap mmioBase="0x40000000">
                <Register name="REG_DAG_CTRL" offset="0x00" access="R/W" width="32"/>
                <Register name="REG_DAG_STATUS" offset="0x04" access="R" width="32"/>
                <Register name="REG_DAG_HEAD_ADDR" offset="0x08" access="R/W" width="64"/>
                <Register name="REG_DAG_GATE_DEC" offset="0x10" access="W" width="32"/>
                <Register name="REG_DAG_IRQ_MASK" offset="0x14" access="R/W" width="32"/>
            </RegisterMap>
        </HardwareDescriptorFormat>
        <CryptographicIntegrity id="POST_QUANTUM_ENCLAVE_INTEGRATION">
            <KeyEncapsulation Standard="NIST FIPS 203" Primitive="ML-KEM-1024" KeySizeBytes="1568"/>
            <DigitalSignature Standard="NIST FIPS 204" Primitive="ML-DSA-87" SignatureSizeBytes="4627"/>
            <HashingAlgorithm Standard="FIPS 202" Primitive="SHA3-256"/>
        </CryptographicIntegrity>
    </NonConventionalListingData>
</ST26SequenceListing>
```

Listing 45.1: WIPO ST.26 XML for DAG Node Taxonomy.

Chapter 46

Source Code: HAL MMIO Driver

```c
#include <stdint.h>
#include <stdbool.h>

#define MMIO_DAG_BASE        0x40000000
#define REG_DAG_CTRL         (*(volatile uint32_t*)(MMIO_DAG_BASE + 0x00))
#define REG_DAG_STATUS       (*(volatile uint32_t*)(MMIO_DAG_BASE + 0x04))
#define REG_DAG_HEAD_ADDR    (*(volatile uint64_t*)(MMIO_DAG_BASE + 0x08))
#define REG_DAG_GATE_DEC     (*(volatile uint32_t*)(MMIO_DAG_BASE + 0x10))

typedef struct __attribute__((packed)) {
    uint32_t task_id;
    uint32_t opcode;
    uint64_t src_phys_addr;
    uint64_t dst_phys_addr;
    uint32_t transfer_bytes;
    uint32_t target_gate_id;
    uint64_t next_descriptor;
} dma_dag_descriptor_t;

void hal_dag_init(uint64_t head_descriptor_phys_addr) {
    REG_DAG_CTRL = 0x02;
    REG_DAG_HEAD_ADDR = head_descriptor_phys_addr;
    REG_DAG_CTRL = 0x01;
}

void hal_dag_atomic_decrement_gate(uint32_t gate_id) {
    REG_DAG_GATE_DEC = gate_id;
}

bool hal_dag_is_busy(void) {
    return (REG_DAG_STATUS & 0x01) != 0;
}
```

Listing 46.1: HAL MMIO register controller (dag_hal.c).

Chapter 47

Source Code: IDE DAG Scheduler (Python)

```
from collections import defaultdict, deque
import threading

class TaskNode:
    def __init__(self, task_id, opcode, payload, affinity=0):
        self.task_id = task_id
        self.opcode = opcode
        self.payload = payload
        self.affinity = affinity
        self.in_degree = 0

class GateNode:
    def __init__(self, gate_id, fan_in):
        self.gate_id = gate_id
        self.counter = fan_in
        self.lock = threading.Lock()

    def decrement(self):
        with self.lock:
            self.counter -= 1
            return self.counter == 0

class DAGEngine:
    def __init__(self):
        self.nodes = {}
        self.edges = defaultdict(list)

    def add_task(self, task_node):
        self.nodes[task_node.task_id] = task_node

    def add_edge(self, src_id, dst_id):
        self.edges[src_id].append(dst_id)
        self.nodes[dst_id].in_degree += 1

    def remove_transitive_edges(self):
        for u in list(self.nodes.keys()):
            for v in list(self.edges[u]):
                queue = deque([w for w in self.edges[u] if w != v])
                while queue:
                    curr = queue.popleft()
                    if curr == v:
                        self.edges[u].remove(v)
                        self.nodes[v].in_degree -= 1
                        break
                    for nxt in self.edges[curr]:
                        queue.append(nxt)

    def execute_topological(self):
        ready_queue = deque([n for n in self.nodes.values() if n.in_degree == 0])
        order = []
        while ready_queue:
            curr = ready_queue.popleft()
            order.append(curr.task_id)
            for neighbor_id in self.edges[curr.task_id]:
                neighbor = self.nodes[neighbor_id]
                neighbor.in_degree -= 1
                if neighbor.in_degree == 0:
```

```
                    ready_queue.append(neighbor)
        return order
```

Listing 47.1: IDE DAG scheduler and transformer.

Part VII

Post-Quantum Cryptographic Integration

Chapter 48

FIPS 203/204/205 Primitives and QRDE-AEAD

48.1 Overview

The DAG hardware execution pipeline integrates NIST FIPS 203 (ML-KEM), FIPS 204 (ML-DSA), and FIPS 205 (SLH-DSA) directly at the descriptor level.

48.2 Mathematical Foundations

48.2.1 Module Learning With Errors (M-LWE)

Let $R = \mathbb{Z}[X]/(X^n + 1)$ with $n = 256$, and $R_q = R/qR$ with $q = 3329$. Given $\mathbf{A} \in R_q^{k \times k}$ and $\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e} \in R_q^k$, M-LWE asserts $(\mathbf{A}, \mathbf{A}\mathbf{s} + \mathbf{e}) \approx_c (\mathbf{A}, \mathbf{u})$ for uniform $\mathbf{u}$.

48.2.2 Module Short Integer Solution (M-SIS)

Given $\mathbf{A} \in R_q^{k \times l}$, find non-zero $\mathbf{z} \in R^l$ such that:

$$\mathbf{A}\mathbf{z} \equiv \mathbf{0} \pmod{q} \quad \text{and} \quad 0 < \|\mathbf{z}\|_\infty \leq \gamma \tag{48.1}$$

48.3 QRDE-AEAD Dual-Envelope Protocol

$$\text{ML-KEM.Encaps}(\text{pk}_{\text{receiver}}) \longrightarrow (c_1, \text{ss}) \tag{48.2}$$

$$\mathbf{K}_{\text{master}} = \text{HKDF-SHA3-384}(\text{ss}, \text{"QRDE-Master-Salt"}) \tag{48.3}$$

$$\mathbf{K}_e = \mathbf{K}_{\text{master}}[0..31], \quad \mathbf{K}_m = \mathbf{K}_{\text{master}}[32..63] \tag{48.4}$$

$$\text{Inner: } ct_2, \tau_2 = \text{AES-256-GCM.Encrypt}(\mathbf{K}_e, \text{IV}, P, \text{AD}_{\text{inner}}) \tag{48.5}$$

$$\text{Outer: } \tau_1 = \text{HMAC-SHA3-256}(\mathbf{K}_m, c_1 \parallel ct_2 \parallel \tau_2 \parallel \text{AD}_{\text{outer}}) \tag{48.6}$$

48.4 Post-Quantum Cryptographic Integration Code (C)

```c
#include <stdint.h>
#include <stdbool.h>

#define MMIO_DAG_BASE           0x40000000
#define REG_DAG_CTRL            (*(volatile uint32_t*)(MMIO_DAG_BASE + 0x00))
#define REG_DAG_HEAD_ADDR       (*(volatile uint64_t*)(MMIO_DAG_BASE + 0x08))
#define REG_DAG_GATE_DEC        (*(volatile uint32_t*)(MMIO_DAG_BASE + 0x10))
#define REG_PQC_ENQUEUE_TRIGGER (*(volatile uint32_t*)(MMIO_DAG_BASE + 0x24))

```

```
#define ML_KEM_1024_BYTES_PUBLIC_KEY   1568
#define ML_KEM_1024_BYTES_CIPHERTEXT   1568
#define ML_DSA_87_BYTES_SIGNATURE      4627

typedef enum {
    OP_DMA_TRANSFER          = 0x01,
    OP_COMPILE_STEP          = 0x02,
    OP_PQC_ML_KEM_1024_ENCAP = 0x10,
    OP_PQC_ML_KEM_1024_DECAP = 0x11,
    OP_PQC_ML_DSA_87_SIGN    = 0x20,
    OP_PQC_ML_DSA_87_VERIFY  = 0x21
} dag_opcode_t;

typedef struct __attribute__((packed)) {
    uint32_t task_id;
    uint32_t opcode;
    uint64_t src_phys_addr;
    uint64_t dst_phys_addr;
    uint64_t key_phys_addr;
    uint32_t payload_bytes;
    uint32_t target_gate_id;
    uint64_t next_descriptor;
} pqc_dag_descriptor_t;

void hal_dag_enqueue_ml_kem_1024_encap(
    pqc_dag_descriptor_t* desc, uint32_t task_id,
    uint64_t pk_addr, uint64_t ct_out_addr, uint64_t ss_out_addr,
    uint32_t gate_id, uint64_t next_desc_addr)
{
    desc->task_id = task_id;
    desc->opcode = OP_PQC_ML_KEM_1024_ENCAP;
    desc->src_phys_addr = pk_addr;
    desc->dst_phys_addr = ct_out_addr;
    desc->key_phys_addr = ss_out_addr;
    desc->payload_bytes = ML_KEM_1024_BYTES_PUBLIC_KEY;
    desc->target_gate_id = gate_id;
    desc->next_descriptor = next_desc_addr;
    REG_PQC_ENQUEUE_TRIGGER = task_id;
}
```

Listing 48.1: PQC HAL MMIO driver (dag_hal_pqc.c).

Part VIII

Manufacturing Designs

Chapter 49

Diamond CVD Fabrication Sequence

1. **Substrate growth**: grow 20 μm CVD diamond on Ir/YSZ/Si(001) handle wafer; lift off.
2. **Backside via etch**: RIE from backside, stop on metal-1 pads; insulate with PECVD SiO_2.
3. **Via metallization**: electroplate Au, 10 μm diameter.
4. **Schottky contact deposition**: Al (100 nm) on phosphorus-doped diamond (5×10^{17} cm^{-3}), patterned by lift-off.
5. **Row/column electrodes**: Ti/Au (10/200 nm) sputtering and lift-off.
6. **H-termination**: hydrogen plasma exposure defines 2D hole gas channels for Q128 core.
7. **Gate dielectric**: Al_2O_3 ALD at 120 °C.
8. **Graphene interconnects**: CVD graphene transfer, patterned.
9. **Glass reservoir cap**: anodic bond at 400 °C with pre-etched 50-μm microchannels.
10. **Parylene-C passivation**: 5 μm CVD, opened at electrode tips and bond pads.
11. **Dice, test, package**: dicing, electrical test, pick-and-place onto flexible inductive coil substrate.

Chapter 50

GDSII Mask Layer Outline

Table 50.1: GDSII mask layer outline.

Layer	Name	Function
1	NWELL	n-type doping (phosphorus)
2	PWELL	p-type doping (boron)
3	HTERM	hydrogen termination regions
4	GATE	Al_2O_3/Ti/Au gate stack
5	SCHOTTKY	Al Schottky contacts
6	METAL1	row electrodes (Ti/Au)
7	METAL2	column electrodes (Ti/Au)
8	GRAPHENE	interconnect layer
9	VIA	through-diamond vias
10	PAD	bond pads
11	PARYLENE_OPEN	electrode tips and bond windows
12	RESERVOIR_ALIGN	glass-bond alignment marks

Chapter 51

Bill of Materials (Device)

Table 51.1: Bill of materials at 10,000 units/year.

Item	Cost (USD)	Notes
20 μm CVD diamond die (4.5×4.5 mm)	60	electronic grade
H-terminated MESFET processing	31	Q128 + multi-ISA decoder incl. LoongArch
VCSEL array (1064 nm, 8 ch)	12	lidar source
PMUT ultrasound array	10	PZT/AlN
Balanced photodetector	15	coherent RX
Waveguide etch	8	shared lidar/opto
Electrophoresis array	12	Pt/Ir, Au vias
Glass reservoir + bond	15	50 μm channels
Parylene-C encapsulation	5	5 μm
Inductive coil + septum	8	power + refill
Nanoparticle reagent (per patient)	10	one-time CSF injection
Total device BOM	**$186**	excl. nano reagent

Chapter 52

Regulatory and Test Flow

Table 52.1: Silicon validation test flow.

Step	Test	Pass Criteria
1	Functional Bring-Up	All layers responsive, no timeout
2	Thermal Throttle Characterization	Z-domain throttles at $\geq$115°C
3	Fault-Injection Campaign	Single-bit corrected $\leq$2 cycles
4	Bayesian Scheduler Validation	Posterior matches Q16.16 model
5	Quantum Emulation Fidelity	Trace distance < 0.01
6	PQC Trust-Chain E2E	Signature verifies, DAG root matches
7	Accessibility Stress Test	Compliance $\geq 95\%$
8	Multi-ISA Workload Tests	Linux, Debian, GCC, Python, Node.js, Firefox, Chromium, PostgreSQL, Loongnix pass

Table 52.2: Regulatory submission plan.

Standard	Scope	Target
IEC 62304	Software lifecycle	Q1 2027
ISO 14971	Risk management	Q1 2027
IEC 60601-2-33	MRI safety	Q2 2027
IEC 60825-1	Laser safety	Q2 2027
ISO 14708-3	Active implants	Q4 2027
IEC 62366-1	Usability	Q2 2027
ISO 10993	Biocompatibility	Q3 2027
Multi-ISA supplemental	Software item annex	Q2 2027
LoongArch supplemental	Software item annex	Q2 2027

Part IX

USPTO Claims (1–28)

Chapter 53

Independent Claims

Claim 1 (System — DAG execution)

1. A system for deterministic Directed Acyclic Graph (DAG) task execution across hardware and software boundaries, the system comprising a physical memory storing a DAG data structure and one or more hardware processing units configured to execute instructions to perform operations comprising: reading a first dependency set from an execution context; generating a primary DAG comprising a plurality of task nodes and gate nodes interconnected by directed edges; transforming the primary DAG into a secondary DAG by applying an algebraic graph transformation that removes transitive redundancy edges and inserts gate nodes at dependency convergence points; identifying immediate task nodes lacking unresolved incoming edges; dispatching immediate task nodes in parallel according to execution affinity masks; and atomically decrementing a target gate counter upon task completion, releasing downstream task nodes when the counter reaches zero.

Claim 2 (Method — DAG build engine)

2. A method for executing a DAG-only build engine within a computing environment, the method comprising: extracting a first dependency set corresponding to software compilation units; constructing a primary DAG comprising task nodes and gate nodes; transforming the primary DAG into a secondary DAG by evaluating transitive dependencies; mapping each task node to a Debug Information Entry (DIE) offset within a DWARF debugging structure; scheduling ready task nodes to a multi-core processing pipeline; and upon detecting a fault in a first task node, isolating the fault and continuing execution of unaffected subgraphs.

Claim 3 (CRM — HAL device driver)

3. A non-transitory computer-readable medium storing instructions that, when executed by at least one hardware processor, cause the processor to execute a HAL device driver model by performing steps comprising: initializing an MMIO controller with a physical memory pointer referencing a root DMA descriptor chain structured as a DAG; writing to a control register to initiate topological DMA processing; receiving a hardware interrupt indicating completion of a first DMA task node; executing an atomic decrement on a fan-in gate counter; and triggering execution of a subsequent DMA task node when the counter reaches zero.

Chapter 54

Open Diamond and Q128 Claims

4. A neural interface system comprising: a single-crystal CVD diamond substrate; a 128-bit RISC-V processor core integrated into said substrate; a frequency-modulated continuous-wave (FMCW) lidar subsystem operating at approximately 1064 nm; a focused ultrasound array operating at 1–5 MHz; a passive-matrix electrophoresis array comprising 144 rows and 233 columns; and a formally verified software stack executing on said processor core; wherein the system is configured to activate neural tissue only by superposition of the lidar field and the ultrasound field onto functionalized nanoparticles distributed via cerebrospinal fluid, and wherein the system contains no electrical stimulation path to tissue.

5. The system of claim 4, wherein the electrophoresis array comprises a diamond Schottky diode at each intersection of a row electrode and a column electrode, and each pixel is controlled by a 128-bit command word comprising a current amplitude field, a pulse waveform field, and a per-pixel safety dose cap.

6. The system of claim 4, wherein the processor core comprises a hardware safety extension fuse-programmed at fabrication and not modifiable by software, and a quantum control extension that directly manipulates NV-centre qubits integrated into the diamond substrate.

7. A method for non-genetic, non-electrical neural activation, comprising: administering functionalized nanoparticles into the cerebrospinal fluid; allowing said nanoparticles to distribute to a target tissue region; simultaneously illuminating said region with an FMCW lidar beam at 1064 nm; simultaneously applying focused ultrasound at 1–5 MHz to said region; verifying that thermal load, mechanical index, and cumulative dose remain within safety limits; wherein the superposition of said fields causes the nanoparticles to release a biochemical ligand, thereby activating neurons without genetic modification and without injecting electrical current into tissue.

8. The method of claim 7, further comprising logging each activation event to an append-only directed acyclic graph whose node hashes incorporate parent hashes, and anchoring the DAG root to write-once storage.

Chapter 55

HSPA Processor Claims (9–18)

9. A processor comprising: a semantic instruction set defining graph-based executable nodes; a probabilistic microcontroller unit configured to apply Bayesian inference to schedule said nodes; a plurality of reconfigurable logic cells; a unified memory lattice encoding physical and semantic address data; and a self-optimizing microcode layer configured to autonomously remap the reconfigurable logic cells based on execution telemetry and to cryptographically verify said remapping against an append-only directed acyclic graph.

10. The processor of claim 9, wherein the probabilistic microcontroller unit comprises a Bayesian inference matrix operating on Q16.16 fixed-point arithmetic and a comparator evaluating posterior confidence against a threshold of `32'h00018000`.

11. The processor of claim 9, wherein the unified memory lattice comprises a 128-bit hybrid address field in which bits [127:64] encode a physical index, bits [63:16] encode a semantic hash, and bits [15:0] encode a privilege and uncertainty vector.

12. The processor of claim 9, further comprising an asymmetric dual-clock hardware transformer core in which 32 Z-latent registers operate at twice a system clock frequency and 16 Y-prediction registers operate at the system clock frequency, and wherein convergence is asserted when a fixed-point L2 norm falls below `32'h0000000A`.

13. The processor of claim 9, further comprising a density-matrix quantum emulation substrate configured to apply Kraus channel operators, enforce Hermiticity, unit trace, and positive semi-definiteness, and emit quantum random numbers via an opcode.

14. The processor of claim 9, further comprising a post-quantum cryptographic trust chain comprising an ML-KEM engine, an ML-DSA hardware signer, and an SLH-DSA signer, wherein signed telemetry units are organized as a directed acyclic graph and the DAG root hash is anchored to write-once storage.

15. A method for cryptographic integrity verification, comprising constructing a directed acyclic graph whose nodes are signed telemetry units, computing a root hash from the unique root node, writing the root hash to write-once storage, and anchoring the root hash to a public blockchain, wherein no Merkle tree is constructed and no Merkle root is computed at any step.

16. The processor of claim 9, further comprising a multi-ISA decode front-end configured to accept instruction bytes in a plurality of legacy binary formats including at least RISC-V RV32I, RV32GC, RV64GC, RV128I, x86-32, x86-64, ARMv7 A32, ARMv7 T32, ARMv8 AArch64, ARMv8 AArch32, LoongArch LA32R, LoongArch LA32S, and LoongArch LA64, and to dispatch the decoded instructions either to a native Q128 semantic graph execution path or to a hardware-assisted dynamic binary translation path.

17. The processor of claim 16, further comprising a cross-ISA register file in which a single physical register file of 128-bit registers is aliased to the architectural register names of each supported ISA, including LoongArch 32 GPRs ($r0–$r31), 32 FPRs ($f0–$f31), and 32 vector registers ($vr0–$vr31 for LSX, $xr0–$xr31 for LASX).

18. The processor of claim 16, further comprising a memory consistency translation engine that converts the memory ordering semantics of the executing legacy ISA into the native Q128 consistency model by inserting or removing fence instructions, including translation of LoongArch DBAR instructions to the equivalent Q128 RVWMO+ fence pattern.

Chapter 56

Extended Claims (19–28)

19. The processor of claim 16, further comprising a syscall dispatcher that accepts syscall invocations in the native ABI of each supported legacy ISA, including LoongArch `syscall 0` with syscall number in $a7 and arguments in $a0–$a6, and translates them to the native Q128 kernel syscall ABI with error-code translation.

20. The processor of claim 16, further comprising a dynamic binary translation cache keyed by guest program counter and guest execution mode, with a configurable associativity between two and eight ways.

21. The processor of claim 16, further comprising a self-modifying code detector with a page protection table, a write trap, an invalidation engine, and a dirty-bit propagation mechanism.

22. A processor comprising: a LoongArch decode front-end configured to accept 32-bit fixed-length LoongArch instructions in LA32R, LA32S, and LA64 variants and to classify them into the 2R, 3R, 4R, 2RI5, 2RI12, 2RI14, 2RI16, 1RI21, and I26 instruction formats; a LoongArch register mapping unit that maps the 32 LoongArch GPRs $r0–$r31 to Q128 physical registers, with $r0 hard-wired to zero, $r1 as link register, $r4–$r11 as argument registers $a0–$a7, $r12–$r20 as temporaries $t0–$t8, and $r23–$r31 as statics $s0–$s8; a LoongArch memory model translator that converts the LoongArch weak consistency model to the Q128 RVWMO+ model, inserting DBAR-equivalent fences and preserving LL/SC semantics including the load/load reorder barrier; and a LoongArch vector translator that maps LSX 128-bit and LASX 256-bit vector instructions to the Q128 vector ALU.

23. The processor of claim 22, wherein the LoongArch memory model translator emits fine-grained DBAR hints for LA664-class processors, decoding bit 4 as ordering versus completion, bit 3 as barrier for previous read, bit 2 as barrier for previous write, bit 1 as barrier for subsequent read, and bit 0 as barrier for subsequent write.

24. The processor of claim 22, further comprising a Loongson Binary Translation (LBT) extension compatibility layer that exposes four scratch registers (scr0–scr3), an x86/ARM `eflags` register, and an x87 FPU stack pointer (`ftop`) to guest binaries that rely on hardware-assisted translation features.

25. The processor of claim 22, wherein the LoongArch syscall dispatcher maps `syscall 0` invocations with the syscall number in $a7 and arguments in $a0–$a6 to the Q128 kernel syscall ABI, and maps the return value from the Q128 ABI back to $a0, clobbering $t0–$t8 as required by the LoongArch ABI.

26. The system of claim 1, wherein the immediate task nodes comprise operations for creating directory frameworks, generating header maps, and copying standard libraries without blocking on unresolved dependencies, and dispatching further comprises evaluating real-time execution constraints using an Earliest Deadline First (EDF) scheduling policy.

27. The method of claim 2, wherein isolating the fault further comprises signaling an IDE user interface to highlight the fault at the exact DWARF DIE offset, and dynamically updating the DMA descriptor chain without halting the MMIO controller.

28. The processor of claim 9, wherein atomic decrements are executed via dedicated bus-locked hardware synchronization instructions and the self-optimizing microcode layer triggers candidate microcode generation when a telemetry count exceeds a threshold of 1024.

Part X

Patent Eligibility and Formal Verification

Chapter 57

Patent Eligibility (35 U.S.C. § 101 / MPEP 2106)

57.1 Step 1 — Statutory Category

Claims 1, 2, and 3 fall within statutory categories: System (Machine), Method (Process), and Non-Transitory Computer-Readable Medium (Manufacture).

57.2 Step 2A, Prong 1 — Abstract Concept

The claims recite technical algorithms for graph transformations ($\mathcal{T}$) and topological execution integrated into concrete technological processes (MMIO register programming, DMA descriptor chaining, CPU affinity mask mapping, DWARF synchronization).

57.3 Step 2A, Prong 2 — Practical Application

- Claims bind task nodes to physical processor affinity masks ($\mathcal{A}$) and physical MMIO register maps (`DAG_HEAD_ADDR`).
- The transformation improves system efficiency by reducing DMA pipeline stalls and isolating fault propagation paths.
- Under MPEP 2106.04(d), improvement of computer hardware pipeline operation constitutes a clear practical application.

57.4 Step 2B — Inventive Concept

1. Hardware-enforced fan-in gate counters coupled to atomic decrement register operations.
2. Dynamic dependency transformation removing transitive edges prior to thread queue insertion.
3. Automatic DWARF DIE offset mapping for real-time interrupt debugging.
4. LoongArch DBAR fine-grained hint translation with LL/SC load/load reorder barrier.
5. DAG-only cryptographic integrity (no Merkle constructions).

Chapter 58

Formal Verification Programme

Table 58.1: Formal verification per layer.

Layer	Method	Tool
ISA specification	Formal semantics	Sail
RTL	Property verification	SymbiYosys
Boot ROM	Absence of runtime errors	SPARK
Kernel	Functional correctness	SPARK
HAL	Type safety + unit tests	Rust + Kani
Drivers	Memory safety	Rust + Kani
Safety monitor	Full functional correctness	SPARK
Therapy application	Functional correctness	Rust + Creusot
CNN	Output bound	α, β-CROWN
Quantum circuits	Equivalence	Qbricks
DAG integrity	Structural proof	Lean 4
Multi-ISA decoder	Per-instruction equivalence	Kani (5,000+ proofs)
Memory model	Litmus conformance	Catt, LISA, RISCV-model, LoongArch litmus
Syscall ABI	Conformance tests	glibc mach, musl, ARM ACT, LoongArch suite
LoongArch decode	Per-instruction equivalence	Kani + LoongArch reference
LoongArch memory model	DBAR + LL/SC litmus	LoongArch litmus suite
Translation cache	Correctness + SMC safety	Kani + litmus
Top-level safety	Mathematical proof	Lean 4

```
theorem dag_integrity
    (g : DAG Node)
    (h_acyclic : IsAcyclic g)
    (h_parent :     n     g, hash n = H (n.data, map hash (parents n)))
    (h_single_root :    ! r, r     g     parents r =    ) :
        g', IsAcyclic g'
      hash (root g) = hash (root g')
      g = g' :=
by
  -- Composed from Kani parent-hash verification and SPARK DAG proofs
  ...

theorem combined_safety
    (p : StimulationPattern)
    (h_verified : SafetyMonitor.approves p = true) :
    ThermalLoad p      <= ThermalLimit
    /\ MechanicalIndex p <= MILimit
    /\ CumulativeDose p  <= DoseLimit
    /\ RACIScore p       >= RACIThreshold :=
by
  -- Composed from SPARK, Kani, alpha-beta-CROWN, Qbricks proofs
  ...
```

Listing 58.1: Lean 4 top-level safety theorem.

Appendix A

Glossary

A32 ARM 32-bit instruction set.

AArch32 ARMv8 processor running in 32-bit mode.

AArch64 ARMv8 64-bit instruction set.

ABI Application Binary Interface.

ALM Adaptive Logic Module.

CAM Content-Addressable Memory.

CSF Cerebrospinal Fluid.

DAG Directed Acyclic Graph.

DBAR LoongArch Data Barrier instruction.

DBT Dynamic Binary Translation.

DVNI Direct Visual Neural Interface.

EDA Electrophoretic Delivery Array.

ENDD Extended Neuromodulation Drug Delivery.

FMCW Frequency-Modulated Continuous-Wave.

HSPA Hybrid Semantic Processing Architecture.

HTC Hardware Transformer Core.

LA32R LoongArch reduced 32-bit variant.

LA32S LoongArch standard 32-bit variant.

LA64 LoongArch 64-bit variant.

LASX Loongson Advanced SIMD Extension (256-bit).

LBT Loongson Binary Translation extension.

LL/SC Load-Linked / Store-Conditional.

LSX Loongson SIMD Extension (128-bit).

MIT Meta-ISA Translator.

ML-DSA Module-Lattice Digital Signature Algorithm.

ML-KEM Module-Lattice Key Encapsulation Mechanism.

MMU Memory Management Unit.

PMCU Probabilistic Microcontroller Unit.

PMUT Piezoelectric Micromachined Ultrasonic Transducer.

PQC Post-Quantum Cryptography.

Q128 128-bit unified processor ISA.

QEXT Quantum extension.

RACI Reasoning–Answer Coherence Index.

RLC Reconfigurable Logic Cell.

RVWMO RISC-V Weak Memory Ordering.

SAFE Hardware safety extension.

SEG Semantic Execution Graph.

SMC Self-Modifying Code.

TMR Triple-Modular Redundancy.

TSO Total Store Order (x86).

UML Unified Memory Lattice.

UOSE Unified Optical Superposition Engine.

VOM Virtual Optical Microdomain.

WORM Write-Once Read-Many.

Appendix B

Complete Figure Index

Table B.1: Complete figure index.

Figure	Title
Lidar DAG	FMCW range–Doppler processing DAG
Nano DAG	CSF nanoparticle administration DAG
Probe X-section	Open Diamond probe cross-section
FIG 1 (HSPA)	HSPA Core System Architecture
FIG 2 (HSPA)	128-Bit SEG Node Bit Structure
FIG 3 (HSPA)	PMCU Bayesian Inference
FIG 4 (HSPA)	Unified Memory Lattice
FIG 5 (HSPA)	SOML Safe-Commit Protocol
FIG 6 (HSPA)	Asymmetric N-Refinement HTC
FIG 7 (HSPA)	LoongDiamond-2.5 Quantum Substrate
FIG 8 (HSPA)	PQC Multi-Tier Trust Chain (DAG)
FIG 9 (HSPA)	ZU9EG PCB Stackup
FIG 10 (HSPA)	HARMONI-X Accessibility Pipeline
FIGS 11–22	Extended Subsystems (RLC, FSM, Bayesian Pipeline, Hamming Router, Testbench, Thermal, Fault Toler
FIG 23	Multi-ISA Decode Front-End
FIG 24	Cross-ISA Register File Aliasing
FIG 25	Memory Consistency Translation
FIG 26	Syscall Dispatcher and ABI Translation
FIG 27	Dynamic Binary Translation Cache
FIG 28	Self-Modifying Code Detection
FIG 29	LoongArch Decode and Execution Path
FIG 30	LoongArch Register Mapping
FIG 31	LoongArch Memory Model and DBAR Translation
FIG 32	LoongArch LSX/LASX Vector Translation
CIT FIG 1	DAG Node Taxonomy and Topology
CIT FIG 2	DAG Dependency Transformer
CIT FIG 3	HAL DAG-Aware DMA Descriptor Chain
CIT FIG 4	IDE/DWARF Integration and Fault Recovery

Bibliography

[1] 35 U.S.C. § 101, § 112.

[2] 37 CFR 1.84.

[3] MPEP 2106 (Subject Matter Eligibility).

[4] FIPS 203 (ML-KEM).

[5] FIPS 204 (ML-DSA).

[6] FIPS 205 (SLH-DSA).

[7] IEEE 754-2019.

[8] A. Waterman and K. Asanović, *The RISC-V Instruction Set Manual*, 2019.

[9] K. Armstrong et al., "ISA Semantics for ARMv8-A, RISC-V, and CHERI-MIPS," POPL 2019.

[10] J. Barnes, *SPARK: The Proven Approach to High Integrity Software*, 2012.

[11] Kani Rust Verifier.

[12] K. Xu et al., "α, β-CROWN," 2021.

[13] C. Chareton et al., "Qbricks," 2020.

[14] L. de Moura and S. Ullrich, "The Lean 4 Theorem Prover," CADE 2021.

[15] Loongson, "Introduction to LoongArch," Linux kernel documentation, 2022.

[16] Loongson, "LoongArch ELF psABI Specification," v1.00.

[17] Loongson, "LoongArch v1.10 DBAR hints," and GCC commit r14-5432, 2023.

[18] Loongson, "Loongson Binary Translation (LBT) extension," Linux kernel KVM patches, 2024.

[19] Cambridge Catt memory model framework.

[20] ARM LISA memory model tool.

[21] IEC 62304:2006/AMD1:2015.

[22] ISO 14971:2019.

[23] ISO 14708-3:2017.

[24] IEC 60601-2-33:2022.

[25] IEC 60825-1:2014.

[26] W3C, WCAG 2.2, 2023.

END OF FILING